

Programació amb esdeveniments

Anton Dalmau Mines

Desenvolupament web en l'entorn client

Índex

Introducció	5
Resultats d'aprenentatge	7
1 Programació amb esdeveniments	9
1.1 Esdeveniments. Gestió d'esdeveniments	9
1.1.1 Gestió dels esdeveniments	10
1.1.2 Propagació d'esdeveniments: Bubbling i Capturing	12
1.2 Programació dinàmica d'esdeveniments	15
1.2.1 Assignar gestors en temps real	15
1.2.2 Passar paràmetres i fer servir el context	16
1.2.3 Delegació d'esdeveniments	17
1.2.4 Cancel·lar esdeveniments i comportaments per defecte	18
2 Validació de formularis	21
2.1 Formularis	21
2.1.1 L'atribut method: GET i POST	21
2.1.2 El paper de JavaScript: interceptar i controlar la tramesa	23
2.1.3 Lectura visual del formulari	25
2.2 Validació de formularis en el cantó del client	30
2.2.1 Tipus d'elements que podem validar	30
2.2.2 Validar amb HTML5 o amb JavaScript	36
2.2.3 Expressions regulars per validar cadenes	39
3 Javascript modern. Programació amb mòduls. Empaquetadors	43
3.1 Programació amb mòduls	43
3.1.1 Importar i exportar mòduls	43
3.1.2 Exportació per defecte	44
3.1.3 Estructura d'un projecte modular	45
3.2 Empaquetadors de Javascript	46
3.2.1 Els empaquetadors més utilitzats	46
3.2.2 Exemple pràctic amb Vite	47
3.2.3 Estructura d'un projecte Vite	47
3.2.4 Per què fem servir un empaquetador?	48
3.2.5 Cas pràctic de projecte amb Vite	49
4 Validació de formularis amb llibreries	55
4.1 Programació d'una llibreria per validar formularis	55
4.1.1 Crear una llibreria bàsica de validació	55
4.1.2 Utilitzar la llibreria	56
4.1.3 Connectar la lògica amb la vista HTML	56
4.1.4 Afegir missatges personalitzats i estil visual	58
4.2 Llibreries per validar formularis	62
4.2.1 Exemple pràctic amb JustValidate	62

4.2.2	Treball modern amb JustValidate	67
5	Framework Express. Formularis: enviament de dades	73
5.1	Framework Express	73
5.1.1	Què és ExpressJS?	74
5.1.2	Per a què serveix i com funciona? Alguns conceptes claus	74
5.1.3	Exemple pas a pas: Mini servidor amb rutes GET/POST	75
5.2	Formularis: enviament de dades al servidor	80
5.2.1	Teoria bàsica: com funciona l'enviament de dades	81
5.2.2	Exemple pràctic: Formulari que envia dades al servidor	81

Introducció

La programació amb esdeveniments és un dels pilars del desenvolupament d'interfícies web interactives, i és l'objectiu principal d'aquesta unitat. En lloc d'executar el codi de manera seqüencial, JavaScript permet respondre a accions de l'usuari o del sistema, com ara clics, moviments del ratolí, pulsacions de teclat o canvis en formularis. En aquesta unitat s'estudia el concepte d'esdeveniment, els diferents tipus d'esdeveniments disponibles i la manera d'associar-los a elements d'una pàgina web mitjançant gestors d'esdeveniments. També es treballa el model de propagació d'esdeveniments (captura i bombolleig) i la gestió de l'objecte *event*. A través d'exemples pràctics, l'alumnat aprendrà a crear aplicacions que reaccionin de manera dinàmica a les accions de l'usuari, millorant l'experiència d'ús. Aquesta unitat és clau per entendre com JavaScript connecta la lògica del programa amb la interacció real en una pàgina web.

Resultats d'aprenentatge

En finalitzar aquesta unitat, l'alumne/a:

5. Desenvolupa aplicacions web interactives integrant mecanismes de maneig d'esdeveniments.

- Reconeix les possibilitats del llenguatge de marques relatives a la captura dels esdeveniments produïts.
- Identifica les característiques del llenguatge de programació relatives a la gestió dels esdeveniments.
- Diferencia els tipus d'esdeveniments que es poden manejar.
- Crea un codi que capturi i utilitzi esdeveniments.
- Reconeix les capacitats del llenguatge relatives a la gestió de formularis web.
- Valida formularis web fent servir esdeveniments.
- Utilitza expressions regulars per facilitar els procediments de validació.
- Prova i documenta el codi.

1. Programació amb esdeveniments

Aquesta lliçó introdueix el concepte d'esdeveniment en JavaScript, un dels fonaments de la interactivitat a les aplicacions web. Els esdeveniments permeten que una pàgina respongui a les accions de l'usuari —com fer clic, escriure text o moure el ratolí— i a canvis en l'estat del document o del navegador. Aprendre com detectar, gestionar i programar aquests esdeveniments de manera eficient, tant de forma estàtica com dinàmica, per aconseguir una interacció fluida i adaptable entre l'usuari i la pàgina web.

1.1 Esdeveniments. Gestió d'esdeveniments

Els **esdeveniments** són un dels pilars fonamentals de la programació en JavaScript al navegador. Ens permeten **detectar i respondre a accions de l'usuari** o del sistema: fer clic a un botó, moure el ratolí, escriure en un camp, carregar una pàgina, prémer una tecla, etc.

Gràcies als esdeveniments, podem **fer que una pàgina web sigui interactiva**, reaccionant de forma dinàmica al que passa a l'entorn.

Un **esdeveniment** és un succés que es produeix en algun element del DOM (un botó, un enllaç, una imatge, etc.), i que el navegador detecta. JavaScript ens dona la possibilitat d'**escollar** aquests esdeveniments i executar una acció quan es produeixen.

El DOM (Document Object Model) és una representació estructurada del contingut d'una pàgina web. Funciona com un **arbre jeràrquic** d'elements, on cada etiqueta HTML (com <div>, <p>, <a>, etc.) es converteix en un objecte que pot ser manipulat mitjançant JavaScript. A la unitat 3 es treballarà en detall el DOM.

Exemples d'esdeveniments habituals:

- click – quan es fa clic amb el ratolí
- mouseover – quan el ratolí passa per sobre
- keydown – quan es prem una tecla
- submit – quan s'envia un formulari
- load – quan es carrega la pàgina
- change – quan canvia el valor d'un camp

Índex d'esdeveniments

Podem trobar una referència a tots els esdeveniments a:
developer.mozilla.org/en-US/docs/Web/Events.

1.1.1 Gestió dels esdeveniments

Per poder reaccionar a un esdeveniment, cal assignar un gestor d'esdeveniments (event handler) a un element del DOM. Hi ha diverses maneres de fer-ho:

A través d'un atribut HTML (no recomanat):

```
1 <button onclick="alert('Hola!')">Fes clic</button>
```

Aquesta és la manera més antiga i menys recomanada, ja que barreja HTML i JavaScript, dificultant la mantenibilitat.

Assignació directa via JavaScript

```
1 let boto = document.getElementById("meuBoto");
2 boto.onclick = function() {
3   alert("Has fet clic!");
4 };
```

Aquesta manera substitueix qualsevol altre onclick anterior. Només es pot assignar **un únic gestor** per cada tipus d'esdeveniment.

Amb addEventListener() (la més moderna i recomanada)

```
1 let boto = document.getElementById("meuBoto");
2 boto.addEventListener("click", function() {
3   alert("Clic capturat amb addEventListener");
4 });
```

Aquesta forma és la més flexible, perquè:

- Permet **afegir múltiples gestors** per un mateix esdeveniment.
- Separa clarament el **comportament del contingut HTML**.
- És compatible amb més funcionalitats com **captura**, **delegació**, etc.

Exemple complet

En aquest exemple crearem una petita pàgina amb diversos elements que responen a diferents tipus d'esdeveniments: un botó que mostra un missatge en fer clic, una caixa que canvia de color quan el ratolí hi passa per sobre, un camp de text que detecta quan s'escriu alguna cosa (keyup) i un altre botó amb resposta al doble clic. L'objectiu és veure com **assignar diferents gestors d'esdeveniments** a diferents elements, i entendre com funciona l'objecte event.

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
```

```
6     <meta name="viewport" content="width=device-width, initial-
      scale=1.0">
7     <title>Esdeveniments</title>
8 </head>
9
10 <body>
11     <button id="btnClick">Fes clic</button>
12     <button id="btnDoble">Doble clic</button>
13     <div id="caixa" style="width:150px; height:150px; background
      :#ddd; margin:10px;"></div>
14     <input type="text" id="campText" placeholder="Escriu alguna
      cosa..." />
15     <p id="missatge"></p>
16
17     <script>
18         // Botó de clic simple
19         document.getElementById("btnClick").addEventListener("
      click", () => {
20             alert("Has fet un clic!");
21         });
22
23         // Botó amb doble clic
24         document.getElementById("btnDoble").addEventListener("
      dblclick", () => {
25             alert("Has fet un doble clic!");
26         });
27
28         // Caixa que canvia de color amb el ratolí
29         const caixa = document.getElementById("caixa");
30
31         caixa.addEventListener("mouseover", () => {
32             caixa.style.backgroundColor = "lightblue";
33         });
34
35         caixa.addEventListener("mouseout", () => {
36             caixa.style.backgroundColor = "#ddd";
37         });
38
39         // Input que detecta quan s'escriu
40         document.getElementById("campText").addEventListener("
      keyup", function (e) {
41             const text = e.target.value;
42             document.getElementById("missatge").textContent = '
      Has escrit: ${text}';
43         });
44
45     </script>
46 </body>
47
48 </html>
```

En aquest exemple veiem repetidament un **patró per gestionar esdeveniments**:

1. Busquem l'element que rebrà l'acció, per exemple `document.getElementById("btnClick")`
2. Li posem una escolta a l'acció que rebrà `.addEventListener("click",`
3. Finalment, quan es produeixi aquest esdeveniment (click), cridarem a una funció (anònima en aquest cas, però no té per què ser sempre així) `, () =>`

```
{ alert("Has fet un clic!"); };
```

Això és així en els exemples dels `<button>` i `<div>` però en el camp de text observem que la funció anònima té un paràmetre que hem anomenat `e`.

Quan definim un gestor d'esdeveniments, **JavaScript envia automàticament** un objecte que representa l'esdeveniment ocorregut. Podem anomenar aquest paràmetre com vulguem: `e`, `evt`, `event`, etc.

Aquest objecte `event` conté **tota la informació sobre el que ha passat**: quin tipus d'esdeveniment és, en quin element s'ha produït, si hi havia tecles especials premudes, la posició del ratolí, etc.

Per tant, repassant l'exemple, veiem que:

- `e` és l'objecte de l'esdeveniment `keyup`.
- `e.target` apunta al camp de text on s'ha escrit.
- `e.target.value` permet llegir el contingut actual del camp de text.

Així, cada vegada que l'usuari escriu una lletra, el navegador sap **quin element** ha generat l'esdeveniment i n'extreu el valor per actualitzar el paràgraf amb el missatge.

1.1.2 Propagació d'esdeveniments: Bubbling i Capturing

Quan es produeix un esdeveniment al navegador (com fer clic en un element), aquest **no només afecta l'element directament implicat**, sinó que pot **propagarse cap amunt o cap avall per l'estructura del DOM**.

Aquesta propagació segueix dues fases principals:

1. **Capturing (captura)**: l'esdeveniment es propaga **des de l'arrel del document (document) cap al node objectiu**.
2. **Bubbling (bombolla)**: després de l'execució en el node objectiu, l'esdeveniment es propaga **cap amunt, des del node objectiu fins al document**.

Aquesta doble fase permet que **diversos elements d'una jerarquia** puguin **escoltar un mateix esdeveniment** en diferents moments de la seva propagació.

Exemple visual

Imaginem aquesta estructura HTML:

```
1 <div id="pare">
2   <button id="fill">Fes clic</button>
3 </div>
```

Quan es fa clic al botó, l'esdeveniment es comporta així:

1. **Captura (*capturing*)**: document → html → body → div#pare → button#fill
2. **Target**: el botó (#fill) rep l'esdeveniment
3. **Bombolla (*bubbling*)**: button#fill → div#pare → body → html → document

Bubbling: comportament per defecte

Quan utilitzem `addEventListener` sense indicar res més, l'esdeveniment es captura en fase de *bubbling*. Això vol dir que **primer s'executa el gestor de l'element més intern**, i després els dels seus “parens”, de dins cap a fora.

A continuació, veurem un exemple de *bubbling*:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Bubbling – comportament per defecte</title>
5   <style>
6     #contenedor { padding: 30px; background: #e5e7eb; border-
7       radius: 10px; }
8     #boto { padding: 10px 16px; }
9   </style>
10 </head>
11 <body>
12   <h2>Bubbling (per defecte)</h2>
13
14   <div id="contenedor">
15     <button id="boto">Fes clic</button>
16   </div>
17
18   <script>
19     const contenedor = document.getElementById('contenedor');
20     const boto = document.getElementById('boto');
21
22     // Listener al botó (fase de bombolla per defecte)
23     boto.addEventListener('click', (e) => {
24       console.log('Clic al botó (bombolla)');
25       // Observa target vs currentTarget
26       console.log('target =', e.target.id, '| currentTarget =',
27         e.currentTarget.id);
28     });
29
30     // Listener al contenedor (fase de bombolla per defecte)
31     contenedor.addEventListener('click', (e) => {
32       console.log('Clic al contenedor (bombolla)');
33       console.log('target =', e.target.id, '| currentTarget =',
34         e.currentTarget.id);
35     });
36   </script>
37 </body>
38 </html>
```

Quan es fa clic al botó, apareix per consola:

```
1 Clic al botó (bombolla)
2 target = boto | currentTarget = boto
3 Clic al contenedor (bombolla)
4 target = boto | currentTarget = contenedor
```

L'esdeveniment **puja** des del botó cap al contenidor.

Capturing: control avançat

Si volem escoltar l'esdeveniment **abans que arribi al seu objectiu final**, podem activar la fase de **captura** afegint `true` com a tercer paràmetre a `addEventListener`:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Capturing – control avançat</title>
5   <style>
6     #contenedor { padding: 30px; background: #e5e7eb; border-
7       radius: 10px; }
8     #boto { padding: 10px 16px; }
9   </style>
10 </head>
11 <body>
12   <h2>Capturing (3r paràmetre = true)</h2>
13
14   <div id="contenedor">
15     <button id="boto">Fes clic</button>
16   </div>
17
18   <script>
19     const contenedor = document.getElementById('contenedor');
20     const boto = document.getElementById('boto');
21
22     // Listener al contenedor en fase de CAPTURA (true)
23     contenedor.addEventListener('click', (e) => {
24       console.log('Capturat pel contenedor (captura)');
25       console.log('target =', e.target.id, '| currentTarget =',
26         e.currentTarget.id);
27     }, true); // <- activa la fase de captura
28
29     // Listener al botó (bombolla per defecte)
30     boto.addEventListener('click', (e) => {
31       console.log('Clic al botó (bombolla)');
32       console.log('target =', e.target.id, '| currentTarget =',
33         e.currentTarget.id);
34     });
35   </script>
36 </body>
37 </html>
```

Ara, la sortida canviarà:

```
1 Capturat pel contenedor (captura)
2 target = boto | currentTarget = contenedor
3 Clic al botó (bombolla)
4 target = boto | currentTarget = boto
```

Primer s'executa el contenedor en **fase de captura**, després el botó en la **fase de bombolla**.

Aturar la propagació

Podem **aturar completament** la propagació d'un esdeveniment amb `event.stopPropagation()` dins del gestor:

```
1 document.getElementById("boto").addEventListener("click", (e) =>
2   {
3     console.log("Només el botó");
4     e.stopPropagation();
5   });
```

Amb això, si es fa clic al botó, l'esdeveniment no arriba al contenidor ni en *bubbling* ni en *capturing*.

Quan és útil cadascuna de les escoltes?

- La **captura** pot ser útil quan volem **interceptar un esdeveniment abans que arribi al seu destí**, com en casos de validació o bloqueig de comportament.
- **bubbling** és ideal per **delegar esdeveniments**: per exemple, en una llista `<ul>`, pots capturar tots els clics dels seus `<li>` des del pare.

1.2 Programació dinàmica d'esdeveniments

Ara que entenem com escoltar esdeveniments, podem entrar en la **programació dinàmica**, que ens permet **crear comportaments complexos i reutilitzables** a partir de com l'usuari interacciona amb la pàgina.

Què vol dir "dinàmica"?

Vol dir que podem **assignar, eliminar o modificar gestors d'esdeveniments en temps real**, segons el context de l'aplicació. També vol dir que podem treballar amb **funcions anònimes, funcions externes**, o fins i tot **passar paràmetres** a través dels esdeveniments.

1.2.1 Assignar gestors en temps real

Podem afegir o treure gestors quan calgui:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Assignar gestors en temps real</title>
5   <style>
6     body { font-family: system-ui, -apple-system, sans-serif;
7           padding: 20px; }
8     button { padding: 10px 16px; border-radius: 8px; border: 1px
9             solid #ccc; cursor: pointer; }
10    #clicable { margin-top: 10px; background-color: #f3f4f6; }
11  </style>
```

```
10 </head>
11 <body>
12
13   <h2>Assignar gestors en temps real</h2>
14
15   <p>
16     Fes clic al botó <strong>Afegir esdeveniment</strong> per
17     activar el comportament del botó "Clicable".
18     Passats 5 segons, l'esdeveniment s'eliminarà automàticament.
19   </p>
20
21   <button id="activar">Afegir esdeveniment</button>
22   <button id="clicable">Clicable</button>
23
24   <p id="info"></p>
25
26   <script>
27     // 1. Seleccionem els elements
28     const btnActivar = document.getElementById('activar');
29     const btnClicable = document.getElementById('clicable');
30     const info = document.getElementById('info');
31
32     // 2. Definim la funció que mostrarà el missatge quan es
33     // faci clic
34     function mostrarMissatge() {
35       console.log('Clic rebut!');
36       info.textContent = 'Has fet clic al botó clicable!';
37     }
38
39     // 3. Afegim el gestor en temps real
40     btnActivar.addEventListener('click', () => {
41       // Afegim l'esdeveniment click al botó "clicable"
42       btnClicable.addEventListener('click', mostrarMissatge);
43       info.textContent = 'Esdeveniment activat! (durant 5 segons)';
44     });
45
46     // Després de 5 segons, eliminem el listener
47     setTimeout(() => {
48       btnClicable.removeEventListener('click', mostrarMissatge);
49       info.textContent = 'Esdeveniment desactivat.';
50     }, 5000);
51   </script>
52 </body>
53 </html>
```

Això és molt útil, per exemple, per **activar** o **desactivar funcionalitats** segons l'estat del programa.

1.2.2 Passar paràmetres i fer servir el context

Com ja vàrem observar en un apartat anterior, els gestors d'esdeveniments reben com a primer paràmetre un objecte especial anomenat `event`, que conté informació sobre què ha passat:


```
1 document.addEventListener("keydown", function(e) {  
2   console.log("Has premut la tecla:", e.key);  
3 });
```

També podem usar funcions que **acceptin arguments addicionals**, usant funcions fletxa o anònimes com a embolcalls:

```
1 function saluda(nom) {  
2   alert("Hola, " + nom);  
3 }  
4  
5 boton.addEventListener("click", () => saluda("Júlia"));
```

1.2.3 Delegació d'esdeveniments

Una tècnica molt poderosa consisteix a **assignar l'esdeveniment a un element pare**, i deixar que aquest gestioni els esdeveniments dels fills. Això és útil quan els elements fills **es generen dinàmicament**, com en una llista que creix amb el temps:

```
1 <html lang="ca">  
2 <head>  
3   <meta charset="utf-8">  
4   <title>Delegació d'esdeveniments</title>  
5   <style>  
6     body {  
7       font-family: system-ui, -apple-system, sans-serif;  
8       padding: 20px;  
9     }  
10    #llista {  
11      list-style: none;  
12      padding: 0;  
13      margin-top: 10px;  
14    }  
15    #llista li {  
16      background: #e5e7eb;  
17      margin: 4px 0;  
18      padding: 8px 12px;  
19      border-radius: 6px;  
20      cursor: pointer;  
21      transition: 0.2s;  
22    }  
23    #llista li:hover {  
24      background: #dbeafe;  
25    }  
26    button {  
27      padding: 8px 12px;  
28      border-radius: 6px;  
29      border: 1px solid #ccc;  
30      cursor: pointer;  
31      background: #f9fafb;  
32    }  
33  </style>  
34 </head>  
35 <body>
```

```

36
37 <h2>Delegació d'esdeveniments</h2>
38
39 <p>
40   En aquest exemple, només el <strong>&lt;ul&gt;</strong> té
      un gestor d'esdeveniments.
41   Cada vegada que fem clic en un element <strong>&lt;li&gt;</strong>, el gestor del pare detecta quin fill s'ha clicat
      gràcies a <em>e.target</em>.
42 </p>
43
44 <button id="afegir">Afegir nova tasca</button>
45
46 <ul id="llista">
47   <li>Tasca 1</li>
48   <li>Tasca 2</li>
49   <li>Tasca 3</li>
50 </ul>
51
52 <script>
53   const llista = document.getElementById('llista');
54   const botoAfegir = document.getElementById('afegir');
55   let comptador = 3; // per numerar les noves tasques
56
57   // Delegació: escoltem un sol cop al <ul>
58   llista.addEventListener('click', function(e) {
59     // Comprovem si el clic ha estat sobre un <li>
60     if (e.target.tagName === 'LI') {
61       alert('Has fet clic a: ' + e.target.textContent);
62     }
63   });
64
65   // Afegir noves tasques (també reaccionaran automàticament)
66   botoAfegir.addEventListener('click', function() {
67     comptador++;
68     const novaTasca = document.createElement('li');
69     novaTasca.textContent = 'Tasca ' + comptador;
70     llista.appendChild(novaTasca);
71   });
72 </script>
73
74 </body>
75 </html>

```

Aquí, encara que afegim nous `<li>` amb JavaScript, **el mateix gestor continuarà funcionant**, perquè està lligat al `<ul>` pare (llista), i fa servir `e.target` per saber exactament què s'ha clicat.

1.2.4 Cancel·lar esdeveniments i comportaments per defecte

Alguns esdeveniments tenen comportaments predeterminats (com enviar un formulari o seguir un enllaç). Podem evitar-ho així:

```

1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">

```

```
4 <title>Cancel·lar esdeveniments i comportaments per defecte</  
  title>  
5 <style>  
6   body {  
7     font-family: system-ui, -apple-system, sans-serif;  
8     padding: 20px;  
9   }  
10  form {  
11    margin-top: 12px;  
12    padding: 12px;  
13    background: #f3f4f6;  
14    border-radius: 8px;  
15    max-width: 400px;  
16  }  
17  input[type="text"] {  
18    padding: 8px;  
19    border-radius: 6px;  
20    border: 1px solid #ccc;  
21    width: 100%;  
22    margin-bottom: 8px;  
23  }  
24  button {  
25    padding: 8px 12px;  
26    border-radius: 6px;  
27    border: 1px solid #ccc;  
28    cursor: pointer;  
29    background: #e5e7eb;  
30  }  
31  a {  
32    color: #2563eb;  
33    text-decoration: underline;  
34  }  
35 </style>  
36 </head>  
37 <body>  
38  
39 <h2>Cancel·lar esdeveniments i comportaments per defecte</h2>  
40  
41 <p>  
42   Per defecte, alguns elements HTML (com els formularis i els  
43   enllaços)  
44   tenen comportaments automàtics: enviar dades o obrir una  
45   nova pàgina.  
46   Amb <em>event.preventDefault()</em> podem <strong>cancel·lar  
47   </strong> aquesta acció i gestionar-la nosaltres amb  
48   JavaScript.  
49 </p>  
50  
51 <h3>Exemple 1: Formulari</h3>  
52 <form id="formulari">  
53   <label for="nom">Nom:</label>  
54   <input type="text" id="nom" placeholder="Escriu el teu nom  
55   ...">  
56   <button type="submit">Enviar</button>  
57 </form>  
58  
59 <p id="missatge"></p>  
60  
61 <h3>Exemple 2: Enllaç</h3>  
62 <p>  
63   <a href="https://www.wikipedia.org" id="enllac">Anar a  
64   Wikipedia</a>  
65 </p>
```

```
60
61 <script>
62   // Formulari: bloquegem l'enviament per defecte
63   const form = document.getElementById('formulari');
64   const missatge = document.getElementById('missatge');
65
66   form.addEventListener('submit', function(e) {
67     e.preventDefault(); // Evita que el navegador envii el
68     // formulari
69     const nom = document.getElementById('nom').value.trim();
70
71     if (nom === '') {
72       missatge.textContent = 'Cal escriure un nom abans d'
73       // enviar.';
74       missatge.style.color = 'red';
75     } else {
76       missatge.textContent = 'Formulari controlat via
77       // JavaScript. Benvingut/da, ${nom}!';
78       missatge.style.color = 'green';
79     }
80   });
81
82   // Enllaç: evitem que redirigeixi i mostrem un missatge
83   // propi
84   const enllac = document.getElementById('enllac');
85
86   enllac.addEventListener('click', function(e) {
87     e.preventDefault(); // Cancel·la el comportament per
88     // defecte (anar a la web)
89     alert('Has clicat l'enllaç, però la navegació ha estat
90     // bloquejada amb preventDefault().');
91   });
92 </script>
93 </body>
94 </html>
```

Això és imprescindible per poder validar dades abans d'enviar-les, o per fer enviaments AJAX, per exemple.

2. Validació de formularis

Els formularis són una de les eines més essencials del desenvolupament web, ja que permeten la interacció directa entre l'usuari i la pàgina. En aquesta lliçó aprendrem com crear, gestionar i validar formularis de manera correcta per garantir que les dades introduïdes siguin coherents, segures i útils. Veurem com combinar les possibilitats que ofereix HTML5 amb la potència de JavaScript per implementar validacions personalitzades i millorar l'experiència d'usuari.

2.1 Formularis

Els **formularis** són la manera estàndard que tenim en l'entorn web per **recollir dades de l'usuari**.

Són una de les parts més importants del desenvolupament web, ja que permeten que la persona que visita una web pugui **enviar informació**: un nom d'usuari, una contrasenya, una opinió, o un fitxer.

En HTML, tot formulari comença amb l'etiqueta `<form>`.

Dins d'aquesta etiqueta col·loquem els **elements del formulari**, que poden ser de diferents tipus:

`<input>`, `<textarea>`, `<select>`, `<button>`, etc.

Estructura bàsica d'un formulari:

```
1 <form action="#" method="POST">
2   <label for="nom">Nom:</label>
3   <input type="text" id="nom" name="nom">
4
5   <label for="email">Correu electrònic:</label>
6   <input type="email" id="email" name="email">
7
8   <button type="submit">Enviar</button>
9 </form>
```

Quan premem el botó **Enviar**, el navegador per defecte intenta **enviar les dades** a la mateixa pàgina o a un URL especificat amb l'atribut `action`.

2.1.1 L'atribut `method`: GET i POST

Els formularis poden enviar les dades de dues maneres principals:

Mètode GET:

- Les dades s'envien **a través de l'URL**.
- És útil per a consultes senzilles o cerques.
- No és segur per enviar contrasenyes o informació personal, ja que les dades són visibles a la barra d'adreces.
- Exemples típics: cercadors, filtres de productes, etc.

```
1 <form action="cercar.php" method="GET">
2   <input type="text" name="q" placeholder="Cerca...">
3   <button type="submit">Cercar</button>
4 </form>
```

Quan enviem aquest formulari, l'URL podria quedar així: <https://exemple.com/cercar.php?q=javascript>

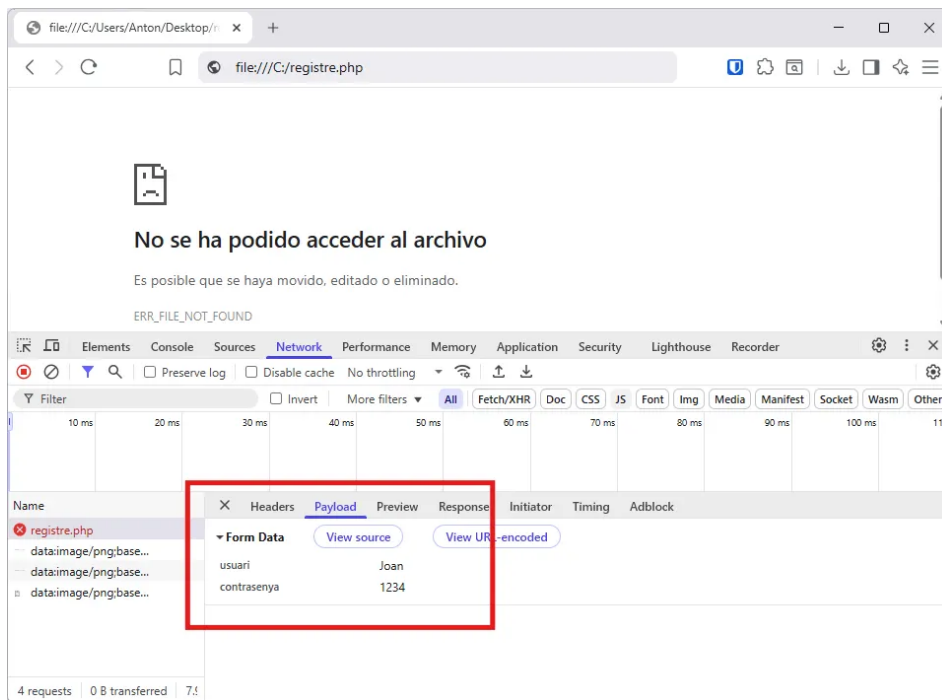
Les dades viatgen a la barra d'adreces, visibles per a qualsevol persona, en aquest cas hem simulat que l'usuari envia el text **javascript**. És pràctic, però no segur.

Mètode POST:

- Les dades s'envien **en el cos (body) de la petició HTTP**.
- No es mostren a l'URL.
- S'utilitza quan volem **enviar dades sensibles** (formularis d'inici de sessió, registres, etc.).
- També permet enviar més quantitat de dades o fitxers.

```
1 <form action="registre.php" method="POST">
2   <input type="text" name="usuari">
3   <input type="password" name="contrasenya">
4   <button type="submit">Registrar</button>
5 </form>
```

En aquest cas, l'URL no canvia, però si obrim l'**Inspector d'elements** → **Xarxa (Network)** (vegeu la figura 5.1), veurem la petició HTTP amb un **payload** com aquest: `usuari=Joan&contrasenya=1234`

FIGURA 2.1. Enviament amb el mètode POST**Visualitzar el cos d'una petició sense servidor real**

Encara que no hi hagi cap servidor real on s'enviïn les dades (per exemple, si estem fent proves locals com en l'exemple anterior on no existeix el recurs `registre.php`), el navegador igualment construeix i envia la petició, i podem veure el cos (*payload*) complet al panell "Network".

2.1.2 El paper de JavaScript: interceptar i controlar la tramesa

Per defecte, un formulari **recarrega la pàgina** quan s'envia.

Això pot no ser desitjat, sobretot si volem fer validacions abans d'enviar-lo o mostrar missatges a l'usuari sense sortir de la pàgina.

Per evitar aquest comportament, utilitzem el mètode:

```
1 event.preventDefault();
```

Aquest mètode "atura" l'acció per defecte del navegador (en aquest cas, l'enviament del formulari).

Sense preventDefault():

```
1 <form id="sensePrevent" onsubmit="enviarFormulari()">
2   <input type="text" name="nom" placeholder="Nom">
3   <button type="submit">Enviar</button>
4 </form>
5
6 <script>
7   function enviarFormulari() {
8     console.log("Formulari enviat!");
```

```
9     }  
10  </script>
```

Quan premem **Enviar** veurem el missatge per consola, però immediatament la pàgina es **recarregarà** i el missatge desapareixerà.

Això és perquè el navegador ha dut a terme el comportament per defecte: enviar el formulari.

En aquest exemple, fem servir l'atribut **onsubmit** dins del formulari per indicar quina funció volem executar quan l'usuari prem el botó **Enviar**.

Això vol dir que, **quan el formulari s'envia**, el navegador crida automàticament la funció `enviarFormulari()`.

Sense `preventDefault()`, però retornant sempre `false`:

```
1  <form id="ambPrevent" onsubmit="interceptarFormulari(); return  
    false;">  
2    <input type="text" name="nom" placeholder="Nom">  
3    <button type="submit">Enviar</button>  
4  </form>  
5  
6  <script>  
7    function interceptarFormulari() {  
8      console.log("Formulari interceptat. La pàgina no es  
        recarrega.");  
9    }  
10 </script>
```

Ara, en prémer el botó, el missatge apareix a la consola i **la pàgina es manté com estava**.

- El formulari té l'atribut `onsubmit="interceptarFormulari(); return false;"`.
- Quan l'usuari prem **Enviar**, el navegador **executa la funció** `interceptarFormulari()`.
- Després, la part `return false` **atura el comportament per defecte del formulari**, és a dir, **evita que la pàgina es recarregui** o que s'enviïn dades.

Això ens permet treballar amb les dades, validar-les o fins i tot enviar-les de forma asíncrona amb `fetch()` (que treballarem més endavant) sense recarregar la web.

Amb `preventDefault()`:

```
1  <form id="ambPrevent" onsubmit="interceptarFormulari(); return  
    false;">  
2    <input type="text" name="nom" placeholder="Nom">  
3    <button type="submit">Enviar</button>  
4  </form>  
5  
6  <script>  
7    function interceptarFormulari() {
```



```
8     console.log("Formulari interceptat. La pàgina no es
      recarrega.");
9   }
10  </script>
```

En aquest exemple quan s'envia el formulari es crida la funció `interceptarFormulari()` que rep l'esdeveniment de l'enviament i gràcies al mètode `preventDefault()` atura el comportament per defecte del formulari.

2.1.3 Lectura visual del formulari

Quan l'usuari omple un formulari, cada camp del formulari conté un **valor** que podem llegir i processar amb JavaScript.

Aquesta lectura ens permet:

- **Mostrar un resum** del que ha introduït l'usuari.
- **Validar** les dades abans d'enviar-les.
- **Manipular-les** o transformar-les per mostrar resultats dinàmics a la web.

Els valors s'obtenen normalment amb la propietat `.value` dels elements `<input>`, `<textarea>` o `<select>`.

Exemple senzill: obtenir valors d'un formulari

En el següent *script* tenim un formulari de petició de dades, de manera que en clicar *Enviar* s'executa la funció `mostrarDades()`:

```
1  <!DOCTYPE html>
2  <html lang="ca">
3  <head>
4    <meta charset="UTF-8">
5    <title>Exemple de formulari</title>
6    <style>
7      form {
8        margin-bottom: 15px;
9        padding: 10px;
10       border: 1px solid #ccc;
11       width: 300px;
12     }
13     label {
14       display: block;
15       margin-top: 10px;
16     }
17   </style>
18 </head>
19 <body>
20
21   <h3>Exemple senzill: obtenir valors d'un formulari</h3>
22
23   <form id="formulari" onsubmit="mostrarDades(); return false;">
24     <label for="nom">Nom:</label>
25     <input type="text" id="nom">
26
27     <label for="email">Correu electrònic:</label>
28     <input type="email" id="email">
29
30     <label for="missatge">Missatge:</label>
31     <textarea id="missatge"></textarea>
32
33     <button type="submit">Enviar</button>
34   </form>
```

```
35
36 <p id="resum"><em>Les dades introduïdes es mostraran aquí.</em></p>
37
38 <script>
39   function mostrarDades() {
40     // Llegim els valors dels camps
41     const nom = document.getElementById("nom").value;
42     const email = document.getElementById("email").value;
43     const missatge = document.getElementById("missatge").value;
44
45     // Mostrem el resultat dins del paràgraf
46     const resultat = document.getElementById("resum");
47     resultat.innerHTML = ''
48     <strong>Dades rebudes:</strong><br>
49     Nom: ${nom}<br>
50     Correu: ${email}<br>
51     Missatge: ${missatge}
52     '';
53   }
54 </script>
55
56 </body>
57 </html>
```

Amb `innerHTML` podem mostrar de manera visual el contingut del formulari dins d'un altre element de la pàgina, com un paràgraf o una caixa de resum.

Llegir valors segons el tipus de camp

Cada tipus d'element pot requerir un tractament diferent. A continuació, mostrem alguns casos habituals:

Camps de text i àrea de text:

```
1 const nom = document.getElementById("nom").value;
2 const comentari = document.getElementById("comentari").value;
```

Aquests camps retornen sempre una cadena de text (`string`).

Podem netejar espais sobrants amb `.trim()`:

```
1 const nomNet = nom.trim();
```

Camps de tipus numèric:

Els inputs numèrics (`type="number"`) retornen igualment un **text** per defecte.

Per convertir-los a número podem fer servir `parseInt()` o `parseFloat()`:

```
1 const edat = parseInt(document.getElementById("edat").value);
```

Si l'usuari deixa el camp buit, `.value` retorna una cadena buida (`""`), no un `0`. Cal **comprovar-ho abans de fer càlculs**.

Radio buttons:

Els radio buttons permeten escollir **una sola opció entre diverses**.

Per obtenir el valor triat, cal buscar quin està **marcat (checked)**.

```
1 <input type="radio" name="genere" value="home"> Home
2 <input type="radio" name="genere" value="dona"> Dona
```

```
1 const genereSeleccionat = document.querySelector("input[name='
  genere']:checked");
2 if (genereSeleccionat) {
3   console.log(genereSeleccionat.value);
4 } else {
5   console.log("No s'ha seleccionat cap opció.");
6 }
```

El mètode `querySelector("input[name='genere']:checked")` retorna l'element seleccionat dins del grup `name="genere"`. Si cap opció està marcada, retorna `null`.

Checkbox:

Les caselles de selecció (checkbox) permeten activar o desactivar opcions.

El seu estat es comprova amb la propietat `.checked`, que retorna `true` o `false`.

```
1 <input type="checkbox" id="condicions"> Accepto les condicions
```

```
1 const condicions = document.getElementById("condicions").checked
  ;
2 if (condicions) {
3   console.log("Condicions acceptades");
4 } else {
5   console.log("Cal acceptar les condicions");
6 }
```

Si tenim **múltiples checkbox amb el mateix nom**, podem recórrer-los amb `querySelectorAll()` i crear un array amb els seleccionats.

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Formulari d'aficions</title>
6   <style>
7     form {
8       padding: 10px;
9       border: 1px solid #ccc;
10      width: 280px;
11      margin-bottom: 15px;
12    }
13    button {
14      margin-top: 10px;
15    }
16  </style>
17 </head>
18 <body>
19
20   <h3>Selecciona les teves aficions</h3>
21
22   <form id="formAficions" onsubmit="mostrarAficions(); return
    false;">
```

```
23     <p>Selecciona les teves aficions:</p>
24
25     <label><input type="checkbox" name="aficions" value="Música
26         "> Música</label><br>
27     <label><input type="checkbox" name="aficions" value="Esport
28         "> Esport</label><br>
29     <label><input type="checkbox" name="aficions" value="Viatges
30         "> Viatges</label><br>
31     <label><input type="checkbox" name="aficions" value="Lectura
32         "> Lectura</label><br>
33
34     <button type="submit">Enviar</button>
35 </form>
36
37 <div id="resumAficions">
38     <em>Les aficions seleccionades es mostraran aquí.</em>
39 </div>
40
41 <script>
42     function mostrarAficions() {
43         // Obtenim totes les caselles marcades
44         const checkboxes = document.querySelectorAll("input[name='
45             aficions']:checked");
46         const aficionsSeleccionades = [];
47
48         // Recorrem els checkbox seleccionats amb un bucle for
49         for (let i = 0; i < checkboxes.length; i++) {
50             aficionsSeleccionades.push(checkboxes[i].value);
51         }
52
53         // Preparem l'element on mostrarem el resultat
54         const resultat = document.getElementById("resumAficions");
55
56         // Mostrem el resultat segons si hi ha selecció o no
57         if (aficionsSeleccionades.length === 0) {
58             resultat.textContent = "No has seleccionat cap afició.";
59         } else {
60             let llista = "<ul>";
61             for (let i = 0; i < aficionsSeleccionades.length; i++) {
62                 llista += "<li>" + aficionsSeleccionades[i] + "</li>";
63             }
64             llista += "</ul>";
65
66             resultat.innerHTML = 'Has seleccionat les següents
67                 aficions: ${llista}';
68         }
69     }
70 </script>
71
72 </body>
73 </html>
```

Llistes desplegable (select):

Les llistes desplegable, definides amb l'etiqueta `<select>`, permeten a l'usuari **escollir una opció d'entre diverses** de manera compacta i ordenada.

Són molt útils quan volem limitar les opcions disponibles (per exemple, seleccionar un país, un curs o un mètode de pagament) i així **evitar errors d'introducció manual**.

Cada opció dins la llista es representa amb l'etiqueta `<option>`, que pot tenir un atribut `value` diferent del text que es mostra visualment.

```
1 <select id="pais">
2   <option value="">Selecciona un país</option>
3   <option value="es">Espanya</option>
4   <option value="fr">França</option>
5 </select>
```

```
1 const pais = document.getElementById("pais").value;
2 console.log("País seleccionat:", pais);
```

Si volem obtenir el text que veu l'usuari (no el `value`), podem fer:

```
1 const select = document.getElementById("pais");
2 const textVisible = select.options[select.selectedIndex].text;
```

Llistes multiselect:

Les llistes multiselect són una variant de les llistes desplegable que permeten **seleccionar més d'una opció alhora**. S'utilitzen quan l'usuari pot tenir múltiples preferències o opcions vàlides (per exemple, seleccionar diversos idiomes o interessos).

Per crear una llista multiselect només cal afegir l'atribut `multiple` a l'etiqueta `<select>`:

```
1 <select id="idiomes" multiple>
2   <option value="ca">Català</option>
3   <option value="es">Castellà</option>
4   <option value="en">Anglès</option>
5   <option value="fr">Francès</option>
6 </select>
```

Per seleccionar diverses opcions en les llistes multiselect, l'usuari pot mantenir premuda la tecla **Ctrl** (Windows/Linux) o **Cmd** (Mac) mentre fa clic.

Com que pot haver-hi **més d'un valor seleccionat**, no podem obtenir-ne un sol `.value`. En comptes d'això, cal recórrer totes les opcions marcades i guardar-ne els valors:

```
1 <form id="formIdiomes" onsubmit="mostrarIdiomes(); return false
2   ;">
3   <label for="idiomes">Selecciona els idiomes que parles:</label>
4   <select id="idiomes" multiple size="4">
5     <option value="ca">Català</option>
6     <option value="es">Castellà</option>
7     <option value="en">Anglès</option>
8     <option value="fr">Francès</option>
9   </select>
10  <button type="submit">Enviar</button>
11 </form>
12 <div id="resumIdiomes">
13   <em>Els idiomes seleccionats es mostraran aquí.</em>
14 </div>
```

```

15
16 <script>
17   function mostrarIdiomes() {
18     const select = document.getElementById("idiomes");
19     const seleccionats = [];
20
21     for (let i = 0; i < select.options.length; i++) {
22       if (select.options[i].selected) {
23         seleccionats.push(select.options[i].text);
24       }
25     }
26
27     const resultat = document.getElementById("resumIdiomes");
28     if (seleccionats.length === 0) {
29       resultat.textContent = "No has seleccionat cap idioma.";
30     } else {
31       resultat.innerHTML = '<strong>Idiomes seleccionats:</strong> ${seleccionats.join(", ")}';
32     }
33   }
34 </script>

```

2.2 Validació de formularis en el cantó del client

La **validació de formularis** garanteix que les dades introduïdes per l'usuari siguin **coherents i segures** abans de processar-les o enviar-les al servidor.

Podem combinar **atributs HTML5** amb **JavaScript** per aconseguir una validació més rica i controlada.

2.2.1 Tipus d'elements que podem validar

Els formularis poden contenir **molts tipus de camps diferents**, i cadascun d'ells serveix per recollir un tipus d'informació concreta.

A la taula 4.1 en veiem alguns dels més habituals:

TAULA 2.1. Tipus d'elements que podem validar

Tipus d'element	Exemple	Funció principal
Text (text)	<code><input type="text"></code>	Introduir text general (nom, cognoms, etc.)
Correu (email)	<code><input type="email"></code>	Assegura que el format sigui de correu electrònic
Nombre (number)	<code><input type="number"></code>	Permet introduir números amb límits mínim i màxim
Àrea de text (textarea)	<code><textarea></textarea></code>	Permet escriure textos llargs o missatges
Selecció única (radio)	<code><input type="radio"></code>	Permet triar una sola opció entre diverses

TAULA 2.1 (continuació)

Tipus d'element	Exemple	Funció principal
Selecció múltiple (checkbox)	<code><input type="checkbox"></code>	Permet activar o desactivar opcions independents
Llista desplegable (select)	<code><select>...</select></code>	Permet triar opcions d'una llista predefinida
Botó (button)	<code><button></button></code>	Envia el formulari o executa accions

N'hi ha molts més tipus (date, color, range, file, tel, etc.), però aquests són els que trobem més sovint en aplicacions web bàsiques i són ideals per practicar la **validació de formularis**.

Cas pràctic: Exemple complet amb validacions amb JavaScript

En aquest formulari inventarem algunes **validacions bàsiques** per comprovar:

- Que el **nom** tingui almenys 3 caràcters.
- Que el **correu** contingui un @ i un punt ..
- Que l'**edat** estigui entre 18 i 99 anys.
- Que s'hagi seleccionat un **gènere**.
- Obtindrem les possibles **aficions** i validarem que almenys n'hi hagi una de seleccionada.
- Que s'hagin **acceptat les condicions**.

Si totes aquestes condicions es compleixen, mostrarem un missatge de validació correcta.

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Formulari complet amb validació</title>
6   <style>
7     form {
8       border: 1px solid #ccc;
9       padding: 15px;
10      width: 320px;
11    }
12    label {
13      display: block;
14      margin-top: 10px;
15    }
16    #missatge {
17      margin-top: 10px;
18      font-weight: bold;
19    }
20  </style>
```

```
21 </head>
22 <body>
23
24   <h3>Formulari complet amb validació bàsica</h3>
25
26   <form id="formulariComplet" onsubmit="validarFormulari();
27     return false;">
28     <label for="nom">Nom complet:</label>
29     <input type="text" id="nom" name="nom">
30
31     <label for="email">Correu electrònic:</label>
32     <input type="email" id="email" name="email">
33
34     <label for="edat">Edat:</label>
35     <input type="number" id="edat" min="18" max="99">
36
37     <p>Gènere:</p>
38     <label><input type="radio" name="genere" value="home"> Home
39       </label>
40     <label><input type="radio" name="genere" value="dona"> Dona
41       </label>
42
43     <label for="aficions">Aficions (pots seleccionar més d'una)
44       </label>
45     <select id="aficions" multiple size="4">
46       <option value="musica">Música</option>
47       <option value="esport">Esport</option>
48       <option value="viatges">Viatges</option>
49       <option value="lectura">Lectura</option>
50     </select>
51
52     <label><input type="checkbox" id="condicions"> Accepto les
53       condicions</label>
54
55     <button type="submit">Enviar</button>
56     <p id="missatge"></p>
57   </form>
58
59   <script>
60     function validarFormulari() {
61       const nom = document.getElementById("nom").value.trim();
62       const email = document.getElementById("email").value.trim
63         ();
64       const edat = document.getElementById("edat").value;
65       const genere = document.querySelector("input[name='genere
66         ']:checked");
67       const condicions = document.getElementById("condicions").
68         checked;
69       const missatge = document.getElementById("missatge");
70       const selectAficions = document.getElementById("aficions")
71         ;
72       const aficionsSeleccionades = [];
73
74       // Obtenim les aficions seleccionades
75       for (let i = 0; i < selectAficions.options.length; i++) {
76         if (selectAficions.options[i].selected) {
77           aficionsSeleccionades.push(selectAficions.options[i].
78             value);
79         }
80       }
81
82       // Validació del nom
83       if (nom.length < 3) {
```



```
74     missatge.textContent = "El nom ha de tenir almenys 3 car
      àcters.";
75     missatge.style.color = "red";
76     return;
77 }
78
79 // Validació del correu electrònic
80 if (!email.includes("@") || !email.includes(".")) {
81     missatge.textContent = "Introdueix un correu electrònic
      vàlid.";
82     missatge.style.color = "red";
83     return;
84 }
85
86 // Validació de l'edat
87 if (edat < 18 || edat > 99 || edat === "") {
88     missatge.textContent = "L'edat ha d'estar entre 18 i 99
      anys.";
89     missatge.style.color = "red";
90     return;
91 }
92
93 // Validació del gènere
94 if (!genere) {
95     missatge.textContent = "Selecciona un gènere.";
96     missatge.style.color = "red";
97     return;
98 }
99
100 // Validació de les aficions
101 if (aficionsSeleccionades.length === 0) {
102     missatge.textContent = "Selecciona almenys una afició.";
103     missatge.style.color = "red";
104     return;
105 }
106
107 // Validació de la checkbox
108 if (!condicions) {
109     missatge.textContent = "Has d'acceptar les condicions
      per continuar.";
110     missatge.style.color = "red";
111     return;
112 }
113
114 // Si tot és correcte
115 missatge.textContent = "Formulari correcte! Totes les
      dades són vàlides.";
116 missatge.style.color = "green";
117 }
118 </script>
119
120 </body>
121 </html>
```

Aquestes validacions no són “oficials” del navegador, sinó **normes que nosaltres inventem amb JavaScript** per comprovar si les dades tenen sentit abans d'enviar-les:

- **Nom:** evitem que l'usuari deixi el camp gairebé buit.
- **Correu electrònic:** comprovem si té la forma d'un correu, tot i que la

validació és molt senzilla (en realitat, els correus poden ser més complexos).

- **Edat:** ens assegurem que sigui un número dins d'un rang raonable.
- **Gènere:** evitem que s'envii sense cap opció marcada.
- **Aficions:** comprovem que l'usuari com a mínim hagi seleccionat una opció.
- **Condicions:** garantim que l'usuari les accepta abans d'enviar.

Aquestes comprovacions són habituals en formularis reals perquè eviten errors i dades incompletes.

Més endavant, quan aprenguem esdeveniments i el mètode `addEventListener()`, escriurem aquest mateix codi d'una manera més professional i separant completament l'HTML del JavaScript. Per ara, però, l'ús de `onsubmit` amb `return false` és **la forma més senzilla i visual** per practicar i entendre com funciona la validació.

Els esdeveniments i el mètode `addEventListener()`, es veuran amb deteniment en la Unitat 3.

Cas pràctic: Exemple de validacions sense ús de Javascript

Amb atributs propis de HTML5 també podem validar formularis de forma semblant a com ho hem fet amb JavaScript.

Anem a veure el mateix exemple i posteriorment analitzem quina és la millor solució:

```

1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Formulari amb validació HTML5</title>
6     <style>
7       form { border: 1px solid #ccc; padding: 15px; width: 320px; }
8       label { display: block; margin-top: 10px; }
9       /* Feedback visual natiu amb CSS */
10      input:required:invalid, textarea:required:invalid, select:
11        required:invalid {
12        border: 2px solid #e57373; /* vermell suau */
13      }
14      input:required:valid, textarea:required:valid, select:
15        required:valid {
16        border: 2px solid #81c784; /* verd suau */
17      }
18      small.hint { color: #666; font-size: 0.9em; }
19    </style>
20  </head>
21  <body>
22    <h3>Formulari complet amb validació HTML5</h3>
23
24    <!-- Sense JS: el navegador valida abans d'enviar -->
25    <form id="formulariComplet" action="#" method="post">
26      <!-- NOM: mínim 3 caràcters, només lletres i espais -->
27      <label for="nom">Nom complet:</label>

```

```
26 <input
27   type="text"
28   id="nom"
29   name="nom"
30   required
31   minlength="3"
32   pattern="^[A-Za-zÀÿ-\s'\-]{3,}$"
33   autocomplete="name"
34   title="Mínim 3 caràcters. Només lletres, espais, apò
      strof i guionet."
35   placeholder="Maria Pujol"
36 />
37 <small class="hint">Ex.: Maria Pujol</small>
38
39 <!-- EMAIL: format de correu -->
40 <label for="email">Correu electrònic:</label>
41 <input
42   type="email"
43   id="email"
44   name="email"
45   required
46   autocomplete="email"
47   inputmode="email"
48   placeholder="exemple@correu.com"
49   title="Introdueix un correu vàlid (exemple@correu.com).\"
50 />
51
52 <!-- EDAT: entre 18 i 99 -->
53 <label for="edat">Edat:</label>
54 <input
55   type="number"
56   id="edat"
57   name="edat"
58   required
59   min="18"
60   max="99"
61   inputmode="numeric"
62   placeholder="18"
63   title="Edat entre 18 i 99.\"
64 />
65
66 <!-- GÈNERE: radio (posem required a un del grup) -->
67 <p>Gènere: <small class="hint">(obligatori)</small></p>
68 <label
69   ><input type="radio" name="genere" value="home" required
      /> Home</label>
70 >
71 <label><input type="radio" name="genere" value="dona" />
      Dona</label>
72
73 <!-- AFICIONS: multiselect obligatori -->
74 <label for="aficions">Aficions (selecciona almenys una):</
      label>
75 <select
76   id="aficions"
77   name="aficions"
78   multiple
79   size="4"
80   required
81   title="Selecciona almenys una afició.\"
82 >
83   <option value="musica">Música</option>
```

```
84         <option value="esport">Esport</option>
85         <option value="viatges">Viatges</option>
86         <option value="lectura">Lectura</option>
87     </select>
88     <small class="hint">Prem Ctrl (o Cmd) per seleccionar més
      d'una.</small>
89
90     <!-- CONDICIONS: checkbox obligatori -->
91     <label style="margin-top:10px;">
92         <input type="checkbox" id="condicions" name="condicions"
      required />
93         Accepto les condicions
94     </label>
95
96     <button type="submit" style="margin-top:10px;">Enviar</
      button>
97 </form>
98
99 <p style="margin-top:12px;">
100     Amb aquesta configuració, si falten camps o tenen format
      incorrecte, el
101     navegador mostrarà missatges d'error nadius i impedirà l'
      enviament fins
102     que tot sigui vàlid.
103 </p>
104 </body>
105 </html>
```

Què hem fet amb HTML5 (sense JavaScript)?

- **required:** fa que el camp sigui obligatori (bloqueja l'enviament si està buit).
- **type="email":** activa la comprovació bàsica de correu.
- **min/max/minlength:** limiten edats i longituds mínimes.
- **pattern:** expressió regular per controlar el format (ex.: nom amb lletres i espais).
- **title:** text d'ajuda que el navegador pot mostrar quan la validació falla.
- **Radio:** n'hi ha prou amb posar **required** a un dels input del grup perquè el navegador exigeixi que n'hi hagi un marcat.
- **Checkbox:** amb **required** forces que s'hagi de marcar per poder enviar.

2.2.2 Validar amb HTML5 o amb JavaScript

Quan treballem amb formularis, tenim **dues grans maneres de validar les dades** abans d'enviar-les:

- fer-ho **amb HTML5**, aprofitant els atributs i comportaments nadius del navegador, o bé

- fer-ho **amb JavaScript**, que ens dona més control i personalització.

Totes dues opcions són vàlides i sovint **es combinen** en projectes reals: primer deixem que HTML5 faci la seva feina bàsica, i després afegim una validació JavaScript per cobrir casos més específics.

Vegem amb detall els avantatges i limitacions de cadascuna.

Validar amb HTML5

Els navegadors moderns ja incorporen un sistema de validació automàtica que s'activa només amb l'ús de determinats **atributs** (required, type, min, max, pattern, step, minlength, maxlength, etc.).

Aquest sistema s'anomena **Constraint Validation API**, i permet bloquejar l'enviament del formulari i mostrar missatges d'error nadius sense necessitat de programar res en JavaScript.

Avantatges:

- **Simplicitat total.** No cal escriure codi JavaScript per comprovar si els camps estan buits o tenen un format incorrecte. Només cal afegir els atributs adequats i el navegador fa la resta. Això ja assegura que l'usuari no pugui enviar el formulari sense complir aquestes condicions.

```
1 <input type="email" required>  
2 <input type="number" min="18" max="99">
```

- **Missatges automàtics.** Els navegadors mostren missatges d'avís personalitzats segons l'idioma del sistema, com ara *"Aquest camp és obligatori"* o *"Introdueix una adreça de correu electrònic vàlida"*. Això redueix l'esforç de desenvolupament i ajuda a mantenir una interfície coherent.
- **Menys codi i menys errors.** En projectes senzills (formularis de contacte, subscripcions, registres bàsics) la validació HTML5 és suficient, segura i estalvia temps. Menys codi vol dir també menys possibilitats d'errors de lògica.
- **Compatibilitat molt bona.** Tots els navegadors moderns (Chrome, Firefox, Edge, Safari, Opera) suporten la majoria d'aquests atributs i comportaments de validació, de manera consistent.
- **Feedback visual natiu.** El navegador també aplica estils visuals com el contorn vermell als camps invàlids o verd als correctes, i permet personalitzar-los amb CSS (:invalid, :valid).

Limitacions:

- **Personalització limitada dels missatges.** Tot i que podem traduir-los parcialment amb `title` o amb l'API de validació (`setCustomValidity()`), els missatges d'error són bàsicament els que decideix el navegador. No podem, per exemple, mostrar un text d'avís al costat d'un camp amb un estil concret sense afegir codi JavaScript.
- **No permet validacions complexes entre camps.** Per exemple, comprovar si la contrasenya i la confirmació coincideixen, o si una data d'inici és anterior a una data de finalització. Aquestes situacions requereixen lògica condicional que només podem fer amb JavaScript.
- **Dependència de l'experiència del navegador.** Cada navegador pot mostrar missatges o icones lleugerament diferents, cosa que pot afectar la coherència visual de la web. Per aplicacions amb disseny molt personalitzat o corporatiu, això pot ser un inconvenient.

Validar amb JavaScript

La validació amb JavaScript és més potent i flexible.

Ens permet controlar completament **quan, com i on** mostrem els missatges d'error, i podem definir **les nostres pròpies regles** sense dependre del navegador.

Avantatges:

- **Personalització total.** Podem crear missatges específics per a cada cas, adaptar-los al llenguatge del projecte o mostrar-los visualment d'una manera atractiva (quadres de colors, icones, etc.). Per exemple:

```
1 if (!email.includes("@")) {  
2   missatge.textContent = "El correu ha de contenir un '@'.";  
3 }
```

- **Control sobre la interfície.** No cal que el navegador mostri els seus missatges nadius: podem decidir quan i com avisar l'usuari. Això ens permet, per exemple, mostrar tots els errors junts al final del formulari o marcar en vermell només els camps concrets amb problemes.
- **Validacions entre camps.** Amb JavaScript podem fer comprovacions que HTML5 no permet, com assegurar que:
 - la contrasenya i la confirmació coincideixin,
 - la data d'inici sigui anterior a la data de fi,
 - o que s'hagi triat almenys una afició d'una llista de checkbox.
- **Integració amb altres funcionalitats.** Podem combinar la validació amb altres comportaments dinàmics: enviar dades amb AJAX, mostrar resums, activar botons només quan tot és correcte, etc.

Limitacions:

- **Més codi, més complexitat.** Encara que és molt més flexible, cal escriure i mantenir el codi manualment. Això pot suposar més feina i més probabilitat d'introduir errors lògics si no es fa amb cura.
- **Sense suport automàtic del navegador.** Si es desactiva JavaScript en el navegador, la validació personalitzada deixa de funcionar. Per això sempre és recomanable **complementar-la amb HTML5** o fer una validació addicional al servidor.
- **Cal més coneixements previs.** Cal entendre conceptes com funcions, condicions, valors buits i propietats dels elements del DOM abans de poder implementar validacions fiables.

2.2.3 Expressions regulars per validar cadenes

Com hem vist anteriorment, dins de la propietat `pattern` dels inputs HTML, hem escrit unes seqüències especials de caràcters, com per exemple:

```
1 pattern="^[A-Za-zÀ-ÿ-\s'\"-]{3,}$"
```

Aquestes seqüències s'anomenen **expressions regulars** (*regular expressions* o **regex**) i serveixen per **definir patrons de text** que permeten comprovar si una cadena (com un nom, correu o contrasenya) compleix un determinat format.

Les **expressions regulars** són molt útils tant a **HTML5** (amb l'atribut `pattern`) com a **JavaScript**, on podem utilitzar-les amb el constructor `RegExp` o la notació `/.../`.

Exemples d'expressions regulars habituals

Una expressió regular és un **patró que descriu una o més cadenes possibles de text**. El navegador o el codi JavaScript compara el text introduït amb aquest patró i decideix si és **vàlid** o **invàlid** (vegeu la taula 2.2).

TAULA 2.2. Expressions regulars habituals

Patró	Què valida
<code>^[A-Za-z]+\$</code>	Només lletres (sense espais ni números)
<code>^[0-9]{4}\$</code>	Exactament 4 dígits (p. ex. un PIN o any)
<code>^[A-Za-zÀ-ÿ-\s'\"-]{3,}\$</code>	Noms amb lletres, espais o guionets (mínim 3 caràcters)
<code>^[\\w\\.-]+@[\\w\\.-]+\\. [A-Za-z]{2,}\$</code>	Format bàsic d'un correu electrònic

Estructura bàsica d'una expressió regular

Una **expressió regular** és una seqüència de caràcters especials que defineixen un **patró de coincidència**. Aquest patró s'utilitza per comprovar si un text compleix una determinada forma o estructura.

Per exemple, podem validar si un camp només conté números, si comença per una lletra majúscula o si un correu té el format correcte.

Les expressions regulars es llegeixen d'**esquerra a dreta** i es componen de **caràcters literals** (que han d'aparèixer tal com són) i **metacaràcters** (que tenen un significat especial dins del patró).

Per això, en una expressió com aquesta:

```
1 ^[A-Za-zÀ-ÿ\s]{3,}$
```

Cada símbol té un **propòsit concret**:

- `^` indica que la coincidència comença al principi del text.
- `[A-Za-zÀ-ÿ\s]` defineix els caràcters permesos (en aquest cas, lletres i espais).
- `{3,}` indica que el patró s'ha de repetir com a mínim tres vegades.
- `$` marca el final de la cadena.

Així, el navegador o el codi JavaScript només acceptarà el text si **tota la cadena** compleix aquest format de principi a fi.

També és important saber que les expressions regulars són **sensibles a majúscules i minúscules** per defecte.

Si volem fer que no ho siguin, podem afegir el **modificador i** (d'*insensitive*). Per exemple:

```
1 const patro = /^[a-z]+$ /i;
```

Aquesta expressió permet tant “hola” com “HOLA”, ja que ignora les majúscules.

Finalment, convé recordar que, tot i la seva aparença complexa, les expressions regulars segueixen una **lògica molt precisa i coherent**. Amb una mica de pràctica, és possible llegir-les com si fossin frases curtes que descriuen patrons de text (“ha de començar per una lletra”, “ha de tenir 5 dígit”, “ha d'acabar amb .com”, etc.).

Això les converteix en una eina potentíssima per validar dades dins de formularis, cercar i reemplaçar text, o analitzar grans quantitats d'informació.

A la taula 2.3, podeu veure els **elements habituals de les expressions regulars**.

TAULA 2.3. Empaquetadors

Element	Significat	Exemple
<code>^</code>	Inici de la cadena	<code>^A</code> → comença per A
<code>\$</code>	Final de la cadena	<code>Z\$</code> → acaba en Z
<code>[]</code>	Conjunt permès de caràcters	<code>[A-Z]</code> → qualsevol lletra majúscula
<code>{n,m}</code>	Repeticions mínim/màxim	<code>[0-9]2,4</code> → entre 2 i 4 dígit
.		

TAULA 2.3 (continuació)

Element	Significat	Exemple
+	Una o més vegades	[a-z]+ → una o més lletres minúscules
?	Opcional (0 o 1 vegada)	colou?r → "color" o "colour"
\d	Qualsevol dígit (0–9)	\d{3} → tres dígits
\s	Espai en blanc	[A-Za-z\s]+ → lletres i espais
.	Qualsevol caràcter	a.b → "acb", "a_b", "a1b"...

Exemples en JavaScript

Anteriorment, hem vist exemples d'expressions regulars dins de l'atribut `pattern` de HTML5, però també podem fer servir expressions regulars directament al nostre codi JavaScript:

```

1  <!DOCTYPE html>
2  <html lang="ca">
3  <head>
4    <meta charset="UTF-8" />
5    <title>Validació amb expressions regulars (JS)</title>
6    <style>
7      form { border: 1px solid #ccc; padding: 15px; width: 320px; }
8      label { display: block; margin-top: 10px; }
9      #missatge { margin-top: 10px; font-weight: bold; }
10   </style>
11 </head>
12 <body>
13
14   <h3>Validació de correu amb RegExp en JavaScript</h3>
15
16   <form id="formCorreu" onsubmit="validarCorreu(); return false;">
17     <label for="email">Correu electrònic:</label>
18     <input type="text" id="email" placeholder="exemple@correu.com">
19
20     <button type="submit" style="margin-top:10px;">Validar</button>
21     <p id="missatge"></p>
22   </form>
23
24   <script>
25     function validarCorreu() {
26       const correu = document.getElementById("email").value.trim();
27       // Patró bàsic de correu: text@text.domini
28       const patroCorreu = /^[w\.-]+@[w\.-]+\.[A-Za-z]{2,}$/;
29
30       const missatge = document.getElementById("missatge");
31
32       if (!patroCorreu.test(correu)) {
33         missatge.textContent = "Introdueix un correu electrònic vàlid.";
34         missatge.style.color = "red";
35         return;
36       }
37
38       missatge.textContent = "Correu vàlid!";
39       missatge.style.color = "green";
40     }
41   </script>
42
43 </body>
44 </html>

```

El mètode `.test(correu)` retorna `true` si la cadena compleix el patró i `false` si no.

3. Javascript modern. Programació amb mòduls. Empaquetadors

Aquesta lliçó explora les bases del JavaScript modern, centrant-nos en la programació modular i en l'ús d'eines actuals per gestionar i optimitzar el codi. Aprendre com dividir una aplicació en mòduls independents per millorar-ne l'organització, la reutilització i el manteniment, i veurem com els empaquetadors —com Vite, Webpack o Parcel— permeten combinar i optimitzar aquests mòduls per preparar projectes eficients i escalables. L'objectiu és comprendre com treballar de manera professional amb JavaScript utilitzant les pràctiques i eines que defineixen el desenvolupament web actual.

3.1 Programació amb mòduls

En els inicis de JavaScript, tot el codi s'escrivia dins d'un únic fitxer `.js`.

A mesura que els projectes van créixer, això va generar problemes:

- Era difícil **organitzar** el codi.
- Les funcions i variables **xocaven** entre fitxers.
- Mantenir grans aplicacions era complicat.

Els mòduls van aparèixer per solucionar tot això. Un **mòdul** és simplement un **fitxer JavaScript independent** que **exporta** parts del seu contingut (funcions, constants, classes, objectes, etc.) perquè puguin ser **importades** des d'altres fitxers.

Aquest sistema **ens permet**:

- Dividir el codi en peces més petites i reutilitzables.
- Mantenir cada part del projecte amb una responsabilitat concreta.
- Evitar conflictes de noms o errors globals.

Un mòdul és com una "caixa" de codi que podem obrir i utilitzar des d'altres fitxers quan la necessitem.

3.1.1 Importar i exportar mòduls

Els mòduls moderns de JavaScript utilitzen les instruccions `export` i `import`, introduïdes amb l'estàndard **ES6 (ECMAScript 2015)**.

Exportar contingut

Podem exportar variables, funcions o classes d'un fitxer per poder-les utilitzar en un altre. Per exemple, imaginem que tenim el fitxer `matematiques.js` amb el següent contingut:

```
1 export const PI = 3.1416;
2
3 export function suma(a, b) {
4   return a + b;
5 }
6
7 export function resta(a, b) {
8   return a - b;
9 }
```

També podem exportar-ho tot junt al final:

```
1 const PI = 3.1416;
2
3 function suma(a, b) {
4   return a + b;
5 }
6
7 function resta(a, b) {
8   return a - b;
9 }
10
11 export { PI, suma, resta };
```

Importar contingut

Un altre fitxer, per exemple `app.js`, pot importar les funcions o constants de `matematiques.js` amb la paraula clau `import`, i tot seguit fer-ne ús:

```
1 import { PI, suma } from './matematiques.js';
2
3 console.log(suma(10, 5)); // 15
4 console.log(PI);          // 3.1416
```

Si qui utilitzarà el mòdul és un fitxer HTML amb el codi incrustat dins les etiquetes `<script>`, la importació **només funcionarà** si el fitxer HTML indica `type="module"` en la importació:

```
<script type="module" src="app.js"></script>
```

3.1.2 Exportació per defecte

Podem definir **una exportació principal** per fitxer amb `export default`.

Això és útil quan un mòdul només té una funció o element principal. Per exemple, imaginem el fitxer `salutacio.js` on només hi ha definida la funció `saluda()`:

```
1 export default function saluda(nom) {  
2   return 'Hola, ${nom}!';  
3 }
```

Ara podríem voler importar-lo a `main.js`. Per importar un *export* per defecte **no cal usar claus**:

```
1 import saluda from './salutacio.js';  
2  
3 console.log(saluda('Anna')); // "Hola, Anna!"
```

Només pot haver-hi **un export default per fitxer**.

3.1.3 Estructura d'un projecte modular

Un projecte amb mòduls pot tenir una estructura així:

```
1 /projecte  
2 |  
3 |-- /src  
4 |   |-- utils.js  
5 |   |-- dades.js  
6 |   |-- main.js  
7 |  
8 |-- index.html
```

El fitxer `utils.js` podria contenir funcions generals, `dades.js` podria tenir informació o classes, i `main.js` serà el punt d'entrada que importarà i coordinarà la resta.

La programació amb mòduls té diverses avantatges:

- **Organització:** el codi queda separat per funcionalitats.
- **Reutilització:** podem importar el mateix mòdul en diversos llocs.
- **Mantenibilitat:** és més fàcil actualitzar o corregir errors.
- **Evitació de conflictes:** cada mòdul té el seu espai de noms (*scope*).
- **Execució controlada:** cada mòdul s'executa només una vegada.

Aquestes són algunes bones pràctiques en programació modular:

- Cada mòdul ha de tenir una **responsabilitat clara** (no s'hauria de posar tot el codi en un sol fitxer).
- Els noms de les funcions han de ser **coherents i descriptius**.

- Usar **exportacions nominals** (`export { ... }`) quan hi ha moltes funcions.
- Usar **exportació per defecte** quan el mòdul només fa una tasca principal.
- Evitar variables globals: declarar tot dins del mòdul o dins de funcions.

3.2 Empaquetadors de Javascript

Quan treballem amb mòduls en un projecte petit, és fàcil mantenir-los: només hem d'afegir `<script type="module">` al nostre HTML.

Però quan el projecte creix i tenim **centenars de fitxers modulars**, el navegador ha de fer moltes peticions separades per descarregar-los tots, cosa que **alenteix molt la càrrega** de la pàgina.

Els **empaquetadors de JavaScript** (*bundlers*) solucionen aquest problema combinant tots els nostres mòduls i dependències en **un únic fitxer optimitzat**, preparat per producció.

Un **empaquetador** agafa tots els teus fitxers `.js`, `.css` i altres recursos, els **analitza, combina i minifica**, i genera un o pocs fitxers optimitzats per al navegador.

3.2.1 Els empaquetadors més utilitzats

En la taula 4.1 mostrem una llista dels empaquetadors disponibles. Nosaltres, en les diferents activitats i exercicis, optarem per Vite.

TAULA 3.1. Empaquetadors

Nom	Característiques principals
Webpack	El més complet i flexible. Permet empaquetar JS, CSS, imatges, fonts i fins i tot HTML. Ideal per a projectes grans.
Parcel	No requereix configuració inicial (<i>zero config</i>). Ideal per començar amb projectes moderns sense complicacions.
Vite	Creat pels autors de Vue. Molt ràpid gràcies a l'ús de mòduls nadius d'ECMAScript i servidor de desenvolupament ultraràpid.
Rollup	Especialitzat en mòduls ES6. Genera fitxers molt nets i lleugers, perfecte per llibreries i components reutilitzables.

3.2.2 Exemple pràctic amb Vite

Suposem que volem crear un projecte modern amb mòduls. Amb Vite, podem fer-ho molt fàcilment:

1. Obrim un terminal i executem:

```
1 npm create vite@latest projecte-vite
```

2. Triem el tipus de projecte (per exemple, “Vanilla” per JavaScript pur).

3. Entrem a la carpeta:

```
1 cd projecte-vite
```

4. Instal·lem les dependències:

```
1 npm install
```

5. I arranquem el servidor local:

```
1 npm run dev
```

Vite utilitza directament els **mòduls ES6** del navegador durant el desenvolupament, sense necessitat d’empaquetar res.

Només crea un **fitxer empaquetat i optimitzat** quan fem el desplegament final (npm run build).

3.2.3 Estructura d’un projecte Vite

Un cop s’ha creat un projecte amb Vite, la seva estructura d’arxius típica pot ser la següent:

```
1 /projecte-vite
2 |
3 |-- /node_modules          # Conté les dependències instal·
   lades
4 |-- /public                # Conté fitxers públics com el
   favicon o el HTML base
5 |   |-- index.html          # Fitxer HTML principal
6 |
7 |-- /src                   # Conté el codi font del projecte
8 |   |-- main.js             # Punt d’entrada de l’aplicació
   JavaScript
9 |   |-- style.css           # Arxiu CSS
10 |
11 |-- package.json           # Conté les dependències del
   projecte i configuració de npm
12 |-- vite.config.js         # Configuració de Vite
```

```
13 | -- /dist          # Conté la versió final del  
    | projecte, optimitzada per producció
```

Tant si utilitzem Vite com qualsevol altre empaquetador trobarem una estructura d'arxius força similar. Aquesta estructura pot variar, sobretot pot créixer si el projecte és gran, però a grans trets sempre veurem els projectes estructurats de forma similar.

Anem a analitzar cadascun dels arxius i carpetes principals:

- **/node_modules:** Aquesta carpeta conté totes les dependències del projecte que han estat instal·lades per **npm**. Aquesta carpeta és creada quan s'executa **npm install** i inclou totes les biblioteques i mòduls que el projecte necessita per funcionar.
- **/public:** Aquesta carpeta conté els arxius públics que el servidor utilitzarà. Generalment, inclou el fitxer **index.html**, que és el punt d'entrada de l'aplicació, així com altres arxius estàtics (com imatges o fonts).
- **/src:** Aquesta carpeta conté els fitxers de codi font de l'aplicació. És on es defineixen els mòduls JavaScript, les funcions d'operació (com les matemàtiques, en el cas de l'exemple), els arxius CSS per a l'estil, etc.
- **package.json:** Aquest fitxer és essencial per a la gestió de dependències de Node.js. Conté les següents seccions:
 - **dependencies:** Les biblioteques i mòduls que el projecte necessita per a funcionar.
 - **devDependencies:** Les eines que es necessiten només durant el desenvolupament (per exemple, Vite o altres eines de compilació).
 - **scripts:** S'hi defineixen comandes com **npm run build** o **npm start**, que són les que es poden executar des de la línia de comandes per realitzar tasques automàtiques com la construcció o el desenvolupament.
- **vite.config.js:** Aquest fitxer conté la configuració específica de Vite per a personalitzar l'entorn de desenvolupament o producció. Aquí es poden definir, per exemple, les rutes d'entrada, optimitzacions, etc.
- **/dist:** Aquesta carpeta és generada quan es fa **npm run build** i conté la versió optimitzada per a producció del projecte. És on es col·loquen els fitxers minificats, empaquetats i llestos per ser servits en un entorn de producció.

3.2.4 Per què fem servir un empaquetador?

Els empaquetadors no només uneixen fitxers, també fan moltes altres tasques automàtiques:

- **Minificació:** eliminen espais i comentaris per reduir la mida dels fitxers.
- **Transpilació:** converteixen codi modern (ES6, ES7, etc.) a versions compatibles amb navegadors antics.
- **Gestió de dependències:** permeten importar llibreries de `node_modules` amb `import`.
- **Hot Reload:** actualitzen automàticament la pàgina quan guardem canvis en el codi.
- **Optimització per producció:** creen fitxers finals més petits, amb nom únic i versions “catxejables”.

3.2.5 Cas pràctic de projecte amb Vite

Per acabar d’entendre bé els punts anteriors cal posar-ho en pràctica, és a dir, crear un projecte des de zero amb un empaquetador i veure la seva utilitat.

Imaginem un **cas pràctic**: volem crear una **calculadora modular amb Vite**.

Suposem la següent **cronologia procedimental**:

1. Crearem un nou projecte calculadora-modular amb Vite.
2. Eliminarem els fitxers que no ens interessin del projecte de mostra i reaprofitarem l’arxiu d’estils `style.css`
3. Crearem tres mòduls:
 - `operacions.js` → que exporta suma, resta, multiplica, divideix.
 - `interficie.js` → que s’encarregarà de llegir els camps d’un formulari i mostrar el resultat. Importarà les operacions de `operacions.js`
 - `main.js` → serà el punt d’entrada que importa `interficie.js` i `style.css`
4. Afegirem un formulari senzill a `index.html` per introduir dos números i triar l’operació.
5. Executarem `npm run dev` i comprovarem el funcionament.
6. Finalment, executa `npm run build` i examinarem com Vite empaqueta tot el projecte.

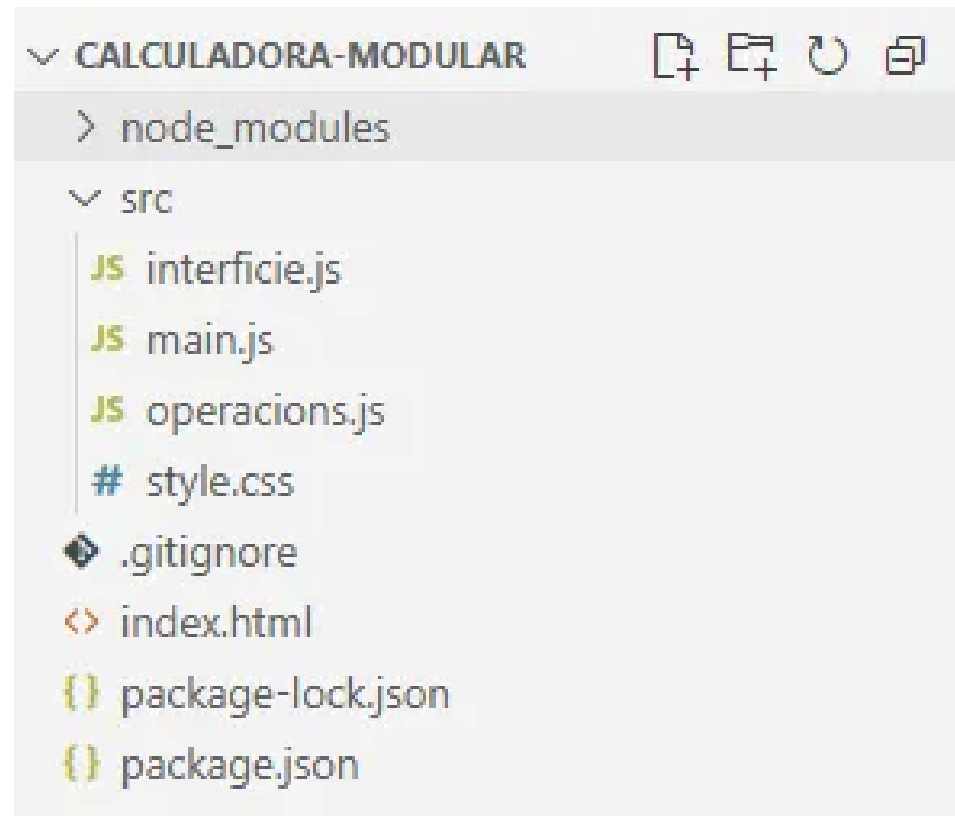
Desenvoluparem el projecte seguint els punts de la cronologia:

1. Crear nou projecte:

```
1 npm create vite@latest calculadora-modular
```

2. L'estructura del projecte pot ser semblant a aquesta (figura 5.1):

FIGURA 3.1. Estructura del projecte



Fitxer style.css:

```
1 :root { line-height: 1.5; }
2 #resultat strong { font-weight: 600; }
```

3. Creació dels mòduls:

Fitxer operacions.js:

```
1 // Funcions pures d'operacions matemàtiques bàsiques
2 export function suma(a, b) { return a + b; }
3 export function resta(a, b) { return a - b; }
4 export function multiplica(a, b) { return a * b; }
5 export function divideix(a, b) {
6   if (b === 0) return "No es pot dividir entre zero.";
7   return a / b;
8 }
```

Fitxer interfície.js:

```
1 import { suma, resta, multiplica, divideix } from "./operacions.
  js";
2
3 function formatNumero(n) {
4   // Evita resultats com 0.30000000000000004
5   const fix = Number.isInteger(n) ? n.toString() : n.toFixed(10)
6   ;
7   return parseFloat(fix).toString();
8 }
```

```
7 }
8
9 export function inicialitzaInterficie() {
10   const form = document.getElementById("calc-form");
11   const aInput = document.getElementById("a");
12   const bInput = document.getElementById("b");
13   const opSelect = document.getElementById("op");
14   const resultat = document.getElementById("resultat");
15
16   form.addEventListener("submit", (e) => {
17     e.preventDefault();
18     const a = parseFloat(aInput.value);
19     const b = parseFloat(bInput.value);
20     const op = opSelect.value;
21
22     try {
23       let valor;
24       switch (op) {
25         case "suma": valor = suma(a, b); break;
26         case "resta": valor = resta(a, b); break;
27         case "multiplica": valor = multiplica(a, b); break;
28         case "divideix": valor = divideix(a, b); break;
29         default: throw new Error("Operació desconeguda");
30       }
31       resultat.textContent = "";
32       resultat.innerHTML = `Resultat: <strong>${formatNumero(
33         valor)}</strong>`;
34     } catch (err) {
35       resultat.textContent = err.message || "S'ha produït un
36         error.";
37     }
38   });
39 }
```

Fitxer main.js:

```
1 import "./style.css";
2 import { inicialitzaInterficie } from "./interficie.js";
3
4 // Inicia l'app quan el DOM està llest
5 if (document.readyState === "loading") {
6   document.addEventListener("DOMContentLoaded",
7     inicialitzaInterficie);
8 } else {
9   inicialitzaInterficie();
10 }
```

4. Creació d'un formulari:

Fitxer index.html:

```
1 <!doctype html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
6       scale=1.0" />
7     <title>Calculadora modular amb Vite</title>
8   </head>
9   <body>
```

```
9      <div id="app" style="max-width: 640px; margin: 2rem auto;
      font-family: system-ui, -apple-system, Segoe UI, Roboto,
      Helvetica, Arial, sans-serif;">
10      <h1>Calculadora modular</h1>
11      <form id="calc-form" style="display:flex; gap:.5rem; flex-
      wrap: wrap; align-items: center;">
12      <input type="number" id="a" required step="any"
      placeholder="Primer número" style="flex:1; min-width:
      8rem; padding:.5rem;">
13      <select id="op" style="padding:.5rem;">
14      <option value="suma">+</option>
15      <option value="resta">-</option>
16      <option value="multiplica">×</option>
17      <option value="divideix">÷</option>
18      </select>
19      <input type="number" id="b" required step="any"
      placeholder="Segon número" style="flex:1; min-width:
      8rem; padding:.5rem;">
20      <button type="submit" style="padding:.6rem 1rem;">
      Calcula</button>
21      </form>
22      <p id="resultat" aria-live="polite" style="margin-top:1rem
      ; font-size:1.125rem;"></p>
23      </div>
24      <script type="module" src="/src/main.js"></script>
25    </body>
26  </html>
```

5. Execució i prova en mode de desenvolupament:

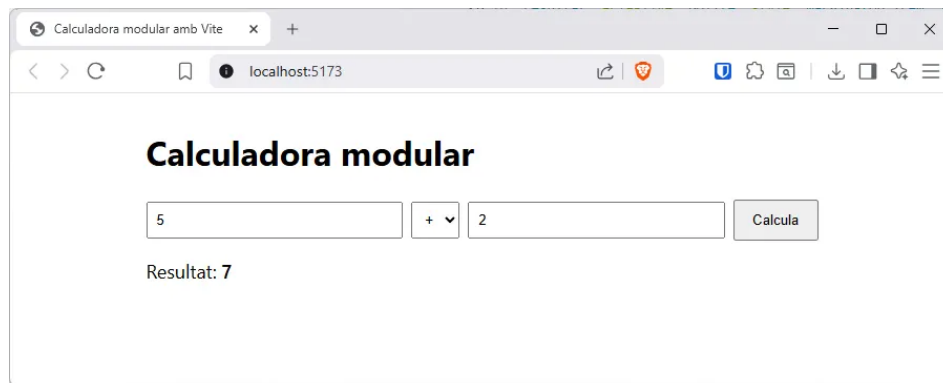
```
1 npm run dev
```

Podem veure el resultat de l'execució en la línia d'ordres (figura 5.4), i obrim el navegador per veure com s'executa (figura 5.2).

FIGURA 3.2. Línia de comandes

```
VITE v7.1.11  ready in 431 ms

→ Local:    http://localhost:5173/
→ Network:  use --host to expose
→ press h + enter to show help
```

FIGURA 3.3. Execució en el navegador

6. Execució i prova en mode de producció:

```
1 npm run build
```

Ja tenim el projecte empaquetat dins la carpeta dist (figura 5.3):

FIGURA 3.4. carpeta "dist/"

Vite ja haurà creat les rutes correctes per trobar els fitxer `.css` i `.js`. Fixem-nos que ha posat uns noms a l'atzar, però l'empaquetador ja haurà creat les rutes correctes al fitxer `index.html`. El projecte ja està llest per ser posat en producció.

Quan es fa el **procés d'empaquetatge** (`npm run build`), l'eina de construcció (com Vite, Webpack, etc.) empaqueta, optimitza i minimitza tots els fitxers del projecte (JavaScript, CSS, imatges, etc.) i els col·loca dins de la carpeta **dist** (abreviatura de *distribution*). Aquest conjunt de fitxers és el que es desplegarà al servidor per ser servit als usuaris.

4. Validació de formularis amb llibreries

En aquesta lliçó, explorarem com millorar la validació de formularis mitjançant l'ús de llibreries externes. Després d'una introducció a la creació d'una llibreria personalitzada per a validacions, aprendrem a utilitzar solucions més avançades com **JustValidate**, una llibreria lleugera i fàcil d'integrar. Aquesta lliçó inclou tant exemples pràctics com la creació d'un formulari complet amb validacions automàtiques, així com el treball amb eines modernes com Vite per millorar l'estructura i la gestió del projecte.

4.1 Programació d'una llibreria per validar formularis

Una **llibreria** és un conjunt de **funcions o mòduls reutilitzables** que resolen un problema concret.

En el cas dels formularis, una llibreria pot ajudar-nos a:

- Centralitzar totes les funcions de validació.
- Reduir codi repetit.
- Mantenir el codi més net i estructurat.
- Reutilitzar les mateixes regles en diferents pàgines o projectes.

A la lliçó 2 vam validar els formularis manualment, camp per camp. Ara crearem una llibreria que ho faci automàticament, passant-li només les regles i els camps.

4.1.1 Crear una llibreria bàsica de validació

Una llibreria no és més que un fitxer `.js` que conté funcions reutilitzables.

Comencem amb una estructura senzilla que encapsulem dins el fitxer `validacions.js`:

```
1 export function campObligatori(valor) {  
2   return valor.trim() !== "";  
3 }  
4  
5 export function longitudMinima(valor, mida) {  
6   return valor.trim().length >= mida;  
7 }  
8  
9 export function correuValid(valor) {  
10   const patron = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
11   return patron.test(valor);  
12 }
```

Aquestes funcions **retornen true o false** segons si el valor és correcte. Són senzilles però fàcilment ampliables.

Aquest fitxer l'hem creat pensant que serà utilitzat en un altre fitxer .js i és per això que hem exportat la seva funcionalitat, gràcies al fet que ja coneixem el treball amb mòduls.

4.1.2 Utilitzar la llibreria

Farem servir la llibreria validacions.js en el fitxer d'entrada de la nostra aplicació main.js:

```
1 import { campObligatori, longitudMinima, correuValid } from './
  validacions.js';
2
3 function validarFormulari() {
4   const nom = document.getElementById("nom").value;
5   const email = document.getElementById("email").value;
6   const missatge = document.getElementById("missatge");
7
8   if (!campObligatori(nom)) {
9     missatge.textContent = "El camp 'nom' és obligatori.";
10    return false;
11  }
12
13  if (!longitudMinima(nom, 3)) {
14    missatge.textContent = "El nom ha de tenir almenys 3 carà
      cters.";
15    return false;
16  }
17
18  if (!correuValid(email)) {
19    missatge.textContent = "El correu no té un format vàlid.";
20    return false;
21  }
22
23  missatge.textContent = "Formulari correcte!";
24  return true;
25 }
```

4.1.3 Connectar la lògica amb la vista HTML

script index.html:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Validació amb llibreria pròpia</title>
6 </head>
7 <body>
8
```



```
9 <h3>Formulari de contacte</h3>
10 <form id="formulari" onsubmit="return validarFormulari();">
11   <label for="nom">Nom:</label>
12   <input type="text" id="nom">
13
14   <label for="email">Correu electrònic:</label>
15   <input type="email" id="email">
16
17   <button type="submit">Enviar</button>
18   <p id="missatge"></p>
19 </form>
20
21 <script type="module" src="main.js"></script>
22 </body>
23 </html>
```

Fixem-nos que:

- Estem treballant amb mòduls ES6.
- El formulari no s'enviarà mai perquè dins de main.js aturem el comportament per defecte amb `event.preventDefault();`. Si volguéssim enviar el formulari després de les validacions senzillament hauríem de canviar la línia `missatge.textContent = "Formulari correcte!"`; per `document.getElementById("formulari").submit();`.

Ja podem testejar el nostre formulari, i com veiem en la figura 5.1 i la figura 5.2 tenim la funcionalitat ben implementada.

També vam treballar amb mòduls ES6 a la lliçó 3.

FIGURA 4.1. Exemple del codi de validació del formulari amb llibreries en el cas que falti algun camp

The screenshot shows a web browser window with the title "Validació amb llibreria pròpia". The address bar shows the URL "127.0.0.1:5500/index.html". The page content includes a heading "Formulari de contacte". Below the heading, there are two input fields: "Nom:" with the value "Joan" and "Correu electrònic:". The "Enviar" button is visible. Below the inputs, a message states "El correu no té un format vàlid." (The email does not have a valid format).

FIGURA 4.2. Exemple del codi de validació del formulari amb llibreries en el cas que la validació sigui correcta

Validació amb llibreria pròpia

127.0.0.1:5500/index.html

Formulari de contacte

Nom: Correu electrònic:

Formulari correcte!

4.1.4 Afegir missatges personalitzats i estil visual

Podem millorar tot el codi anterior per tal que els missatges d'error ens sortissin just sota l'element equivocat amb un estil visual adequat per millorar l'experiència de l'usuari.

Al fitxer `index.html` definirem uns estils i crearem unes etiquetes ocultes amb els **missatges d'error**:

```

1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Formulari amb validació (mòduls)</title>
6     <meta name="viewport" content="width=device-width, initial-
7       scale=1" />
8     <style>
9       form { max-width: 420px; margin: 2rem auto; font-family:
10         system-ui, sans-serif; }
11       label { display: block; margin-top: 1rem; font-weight:
12         600; }
13       input { width: 100%; padding: .6rem; margin-top: .35rem;
14         border: 1px solid #bbb; border-radius: .5rem; }
15       button { margin-top: 1rem; padding: .7rem 1rem; border: 0;
16         border-radius: .6rem; cursor: pointer; }
17       .error { display: block; margin-top: .25rem; }
18     </style>
19   </head>
20   <body>
21     <h3 style="text-align:center">Formulari de contacte</h3>
22
23     <form id="formulari">
24       <div>
25         <label for="nom">Nom</label>
26         <input type="text" id="nom" name="nom" />
27         <span
28           class="error"

```

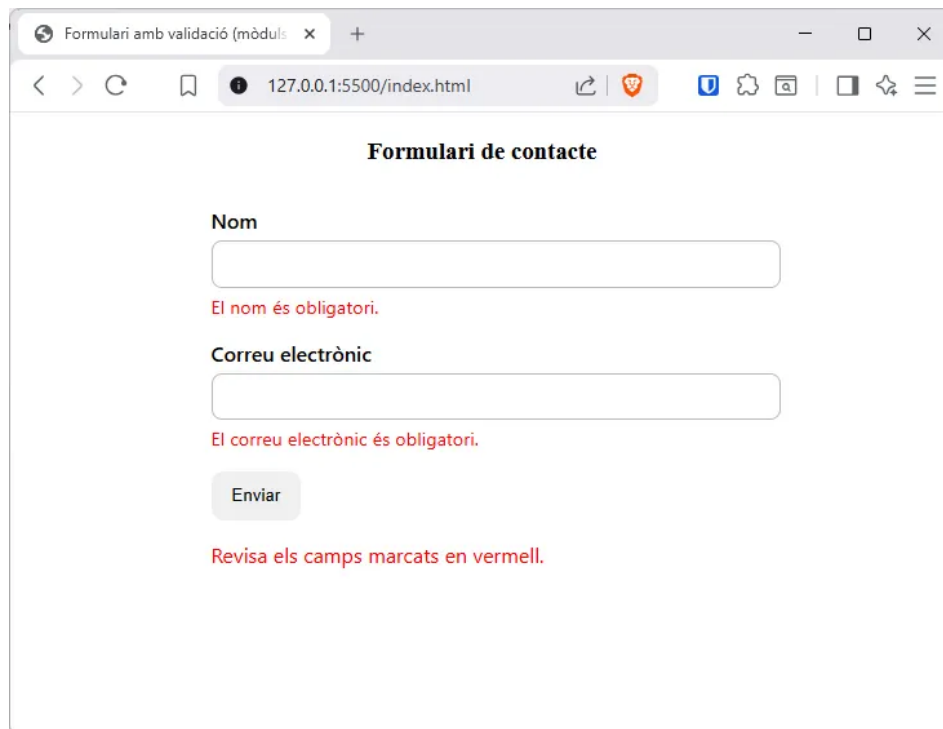
```
24         style="color: red; font-size: 0.9rem; display: none;"
25         >El nom és obligatori.</span>
26     >
27 </div>
28
29 <div>
30     <label for="email">Correu electrònic</label>
31     <input type="email" id="email" name="email" />
32     <span
33         class="error"
34         style="color: red; font-size: 0.9rem; display: none;"
35         >El correu electrònic és obligatori.</span>
36     >
37 </div>
38
39 <button type="submit">Enviar</button>
40 <p id="missatge"></p>
41 </form>
42
43 <script type="module" src="./main.js"></script>
44 </body>
45 </html>
```

I ampliarem l'arxiu main.js afegint els canvis:

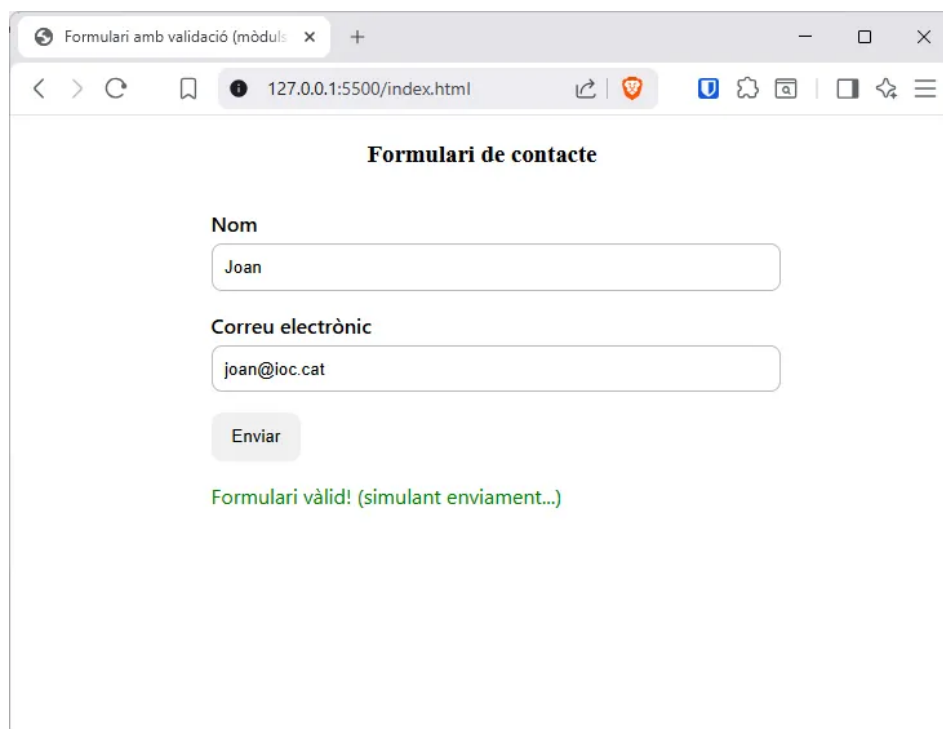
```
1 import { campObligatori, longitudMinima, correuValid } from './
  validacions.js';
2
3 function mostraError(input) {
4     let span = input.parentElement.querySelector('.error');
5     span.style.display = 'block';
6 }
7
8 function netejaError(input) {
9     const span = input.parentElement.querySelector('.error');
10    span.style.display = 'none';
11 }
12
13 function validaCampNom(input) {
14     const valor = input.value;
15     if (!campObligatori(valor)) {
16         mostraError(input);
17         return false;
18     }
19     if (!longitudMinima(valor, 2)) {
20         mostraError(input);
21         return false;
22     }
23     netejaError(input);
24     return true;
25 }
26
27 function validaCampEmail(input) {
28     const valor = input.value;
29     if (!campObligatori(valor)) {
30         mostraError(input, 'El correu és obligatori.');
```

```
36   }
37   netejaError(input);
38   return true;
39 }
40
41 const form = document.getElementById('formulari');
42 const nom = document.getElementById('nom');
43 const email = document.getElementById('email');
44 const missatge = document.getElementById('missatge');
45
46 // Validació en esdeveniments de sortida/canvi per millorar UX
47 // Quan l'usuari surt del camp, el validem
48 nom.addEventListener('blur', () => validaCampNom(nom));
49 email.addEventListener('blur', () => validaCampEmail(email));
50
51 form.addEventListener('submit', (e) => {
52   e.preventDefault(); // evita l'enviament automàtic
53
54   const nomOK = validaCampNom(nom);
55   const emailOK = validaCampEmail(email);
56
57   if (nomOK && emailOK) {
58     missatge.textContent = 'Formulari vàlid! (simulant
59       enviament...)';
60     missatge.style.color = 'green';
61
62     nom.netejaError();
63     email.netejaError();
64   } else {
65     missatge.textContent = 'Revisa els camps marcats en
66       vermell.';
67     missatge.style.color = 'red';
68   }
69 });
```

La nostra llibreria `validacions.js` com que no té cap relació amb la vista no pateix cap canvi respecte a la versió del punt anterior. El comportament del formulari després d'afegir la capa d'estils el podem veure en la figura 5.3 i la figura 5.4.

FIGURA 4.3. Formulari amb camps obligatoris. Missatge d'error

The screenshot shows a web browser window with the title "Formulari amb validació (mòdul: x)". The address bar shows "127.0.0.1:5500/index.html". The page content is titled "Formulari de contacte". It contains two input fields: "Nom" and "Correu electrònic". Below the "Nom" field, there is a red error message: "El nom és obligatori." Below the "Correu electrònic" field, there is a red error message: "El correu electrònic és obligatori." Below these fields is a button labeled "Enviar". At the bottom of the form, there is a red message: "Revisa els camps marcats en vermell."

FIGURA 4.4. Es compleixen les regles de validació

The screenshot shows the same web browser window as Figure 4.3. The "Nom" field now contains the text "Joan" and the "Correu electrònic" field contains the text "joan@ioc.cat". The "Enviar" button is still present. Below the button, there is a green message: "Formulari vàlid! (simulant enviament...)"

Fixem-nos que per aconseguir aquest estil visual hem hagut de manipular manualment el DOM, mostrant i ocultant etiquetes, que és quelcom que estudiarem amb més calma més endavant.

Aquesta estructura és **molt flexible i pot créixer** fins a convertir-se en una llibreria completa de validació per reutilitzar-la en altres projectes.

4.2 Llibreries per validar formularis

Quan el projecte creix, escriure totes les validacions manualment pot resultar feixuc.

Per això, hi ha **llibreries ja preparades** que ens ajuden a validar formularis de manera més ràpida i consistent. A la taula 4.1 podeu veure algunes de les més conegudes:

TAULA 4.1. Llibreries de validació de formularis

Llibreria	Característiques principals
JustValidate	Lleugera, sense dependències, molt fàcil d'integrar en projectes moderns.
Parsley.js	Proporciona validació declarativa amb atributs HTML i missatges personalitzables.
Validator.js	Centrada en validar valors (emails, telèfons, dates, números, etc.) dins del JS, no directament al DOM.
Yup	Sovint usada amb React i <i>frameworks</i> moderns per validar objectes complexos.
Formik + Yup	Combinació molt utilitzada per gestionar i validar formularis en React.

4.2.1 Exemple pràctic amb JustValidate

Escollim **JustValidate** com a llibreria de validació de formularis, i amb ella farem un exemple pràctic. El procés serà el següent:

1. Afegir la llibreria
2. Crear el formulari
3. Configurar la validació
4. Resultat final

Afegir la llibreria

Podem importar **JustValidate** des de CDN al nostre HTML:

```
1 <script src="https://cdn.jsdelivr.net/npm/just-validate@latest/dist/just-validate.production.min.js"></script>
```

Un **CDN** (*Content Delivery Network*) és una xarxa de servidors distribuïts geogràficament que permet que els fitxers es carreguin més ràpidament des del servidor més proper a la ubicació de l'usuari.

Crear el formulari

```
1 <form id="formulariContacte">
2   <div class="form-group">
3     <label for="nom">Nom complet:</label>
4     <input type="text" id="nom" name="nom" />
5   </div>
6   <div class="form-group">
7     <label for="email">Correu electrònic:</label>
8     <input type="email" id="email" name="email" />
9   </div>
10  <div class="form-group">
11    <label for="edat">Edat:</label>
12    <input type="number" id="edat" name="edat" />
13  </div>
14  <button type="submit">Enviar</button>
15 </form>
16
17 <p id="missatge"></p>
```

Configurar la validació

Quan usem una llibreria com **JustValidate**, no cal que nosaltres escrivim manualment totes les funcions per comprovar si un camp està buit, si té prou caràcters o si un correu té el format correcte.

La llibreria ja ens ofereix **regles predefinides** que podem aplicar fàcilment a cada camp del formulari.

Aquí tenim el codi:

```
1 <script>
2   const validator = new JustValidate('#formulariContacte');
3
4   validator
5     .addField('#nom', [
6       { rule: 'required', errorMessage: 'El nom és obligatori' },
7       { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 caràcters' },
8     ])
9     .addField('#email', [
10      { rule: 'required', errorMessage: 'El correu és obligatori' },
11      { rule: 'email', errorMessage: 'El correu no és vàlid' },
12    ])
13     .addField('#edat', [
14      { rule: 'required', errorMessage: 'L'edat és obligatòria' },
15    ]
16  );
```

```
15     { rule: 'number', errorMessage: 'Només es permeten números  
16         ' },  
17     { rule: 'minNumber', value: 18, errorMessage: 'Has de ser  
18         major d'edat' },  
19     ])  
20     .onSuccess(() => {  
21         document.getElementById("missatge").textContent = "  
    Formulari correcte!";  
    });  
</script>
```

Abans d'escriure el codi anterior, ens hem hagut de preguntar:

1. Quin formulari volem validar?

Volem validar al formulari amb *id* `formulariContacte`. Això ho aconseguim creat l'objecte:

```
1 const validator = new JustValidate('#formulariContacte');
```

2. Quins camps del formulari volem comprovar?

- Nom
- Email
- Edat

3. Quines regles s'han d'aplicar a cada camp?

- Pel camp **nom** volem que:
 - Existeixi.
 - Tingui una longitud mínima de 3 caràcters.
- Pel camp **email** volem que:
 - Existeixi.
 - Tingui el format que **JustValidate** consideri que ha de tenir una adreça de correu electrònic.
- Pel camp **edat** volem que:
 - Existeixi.
 - Sigui de tipus numèric.
 - El valor sigui més gran que 18.

4. Quins missatges d'error s'han de mostrar si no es compleix una regla?

Cada regla té l'atribut `errorMessage` on definim el missatge en cas que no es compleixi la regla.

Aquesta llibreria detecta automàticament els errors i mostra els missatges al costat dels camps sense necessitat d'escriure gaire codi.

Resultat final

Si posem tot el codi anterior en un sol fitxer .html i afegim una mica d'estil tenim el següent codi:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7     <!-- Importar JustValidate des de CDN -->
8     <script src="https://cdn.jsdelivr.net/npm/just-
      validate@latest/dist/just-validate.production.min.js"></
      script>
9   <style>
10    /* Estils bàsics per al formulari i els missatges d'error
      */
11    body {
12      font-family: Arial, sans-serif;
13      margin: 20px;
14    }
15    form {
16      max-width: 400px;
17      padding: 20px;
18      border: 1px solid #ccc;
19      border-radius: 8px;
20    }
21    .form-group {
22      margin-bottom: 15px;
23    }
24    label {
25      display: block;
26      margin-bottom: 5px;
27      font-weight: bold;
28    }
29    input[type="text"],
30    input[type="number"],
31    input[type="email"] {
32      width: 100%;
33      padding: 10px;
34      border: 1px solid #ccc;
35      border-radius: 4px;
36      box-sizing: border-box; /* Important per incloure
      padding al width */
37    }
38    button {
39      background-color: #4CAF50;
40      color: white;
41      padding: 10px 15px;
42      border: none;
43      border-radius: 4px;
44      cursor: pointer;
45    }
46    /* Estil per als missatges d'error de JustValidate */
47    .js-validate-error-label {
48      color: red;
49      font-size: 0.9em;
50      margin-top: 5px;
51    }
52  </style>
```

```
53 </head>
54 <body>
55   <div class="container">
56     <h3>Formulari de Contacte</h3>
57     <form id="formulariContacte">
58       <div class="form-group">
59         <label for="nom">Nom complet:</label>
60         <input type="text" id="nom" name="nom" />
61       </div>
62       <div class="form-group">
63         <label for="email">Correu electrònic:</label>
64         <input type="email" id="email" name="email" />
65       </div>
66       <div class="form-group">
67         <label for="edat">Edat:</label>
68         <input type="number" id="edat" name="edat" />
69       </div>
70       <button type="submit">Enviar</button>
71     </form>
72     <p id="missatge"></p>
73   </div>
74
75   <script>
76     // Inicialitzar JustValidate
77     const validator = new JustValidate('#formulariContacte', {
78       // Configuració opcional per localitzar o canviar l'
79       // estil
80       errorFieldCssClass: 'is-invalid',
81       errorLabelCssClass: 'js-validate-error-label',
82     });
83
84     // Afegir regles de validació per als camps
85     validator
86       .addField('#nom', [
87         { rule: 'required', errorMessage: 'El nom és
88           obligatori' },
89         { rule: 'minLength', value: 3, errorMessage: 'Mínim 3
90           caràcters' },
91       ])
92       .addField('#email', [
93         { rule: 'required', errorMessage: 'El correu és
94           obligatori' },
95         { rule: 'email', errorMessage: 'El correu no és vàlid'
96           },
97       ])
98       .addField('#edat', [
99         { rule: 'required', errorMessage: "L'edat és obligatò
100           ria" },
101         { rule: 'number', errorMessage: 'Només es permeten nú
102           meros' },
103         { rule: 'minNumber', value: 18, errorMessage: "Has de
104           ser major d'edat" },
105       ])
106       .onSuccess(() => {
107         // Mostrar missatge d'èxit quan totes les validacions
108         // són correctes
109         document.getElementById("missatge").textContent = "
110           Formulari correcte!";
111         document.getElementById("missatge").className = "
112           success-message";
113       });
114   </script>
115 </body>
```

```
105 </html>
```

Ens hem de fixar que hem afegit algun detall més al constructor de l'objecte validator per tal que ens mostri els errors sota el camp input que conté l'error:

```
1 const validator = new JustValidate("#formulariContacte", {  
2   // Configuració opcional per localitzar o canviar l'estil  
3   errorFieldCssClass: "is-invalid",  
4   errorLabelCssClass: "js-validate-error-label",  
5 });
```

I amb el codi anterior aconseguim un formulari semblant al de la figura 5.5.

FIGURA 4.5. Formulari de contacte amb camps obligatoris

Llibreria JustValidate

Podem trobar la documentació de la llibreria JustValidate a just-validate.dev.

Formulari de Contacte

El formulari de contacte està presentat dins d'un container amb una vora arrodonida i una ombra. Conté tres camps d'entrada, cadascun amb un label, un input i un missatge d'error en vermell. El primer camp és 'Nom complet:' amb el missatge 'El nom és obligatori'. El segon camp és 'Correu electrònic:' amb el missatge 'El correu és obligatori'. El tercer camp és 'Edat:' amb el missatge 'L'edat és obligatòria'. A la part inferior esquerra hi ha un botó verd amb el text 'Enviar'.

4.2.2 Treball modern amb JustValidate

Ara reescriurem l'exemple del punt anterior, però des d'una aproximació més moderna, aprofitant l'aprenentatge d'ús d'empaquetadors de la lliçó anterior. El procés serà el següent:

1. Crear un nou projecte amb Vite
2. Instal·lació de JustValidate
3. Creació del formulari i les regles de validació

4. Execució del projecte

Crear un nou projecte amb Vite

Crearem un nou projecte amb **Vite** i utilitzem la plantilla **vanilla**:

```
1 npm create vite@latest projecte-justvalidate
```

Si no ens ho fa sol, llençarem `npm install` per tenir les dependències instal·lades.

Instal·lació de JustValidate

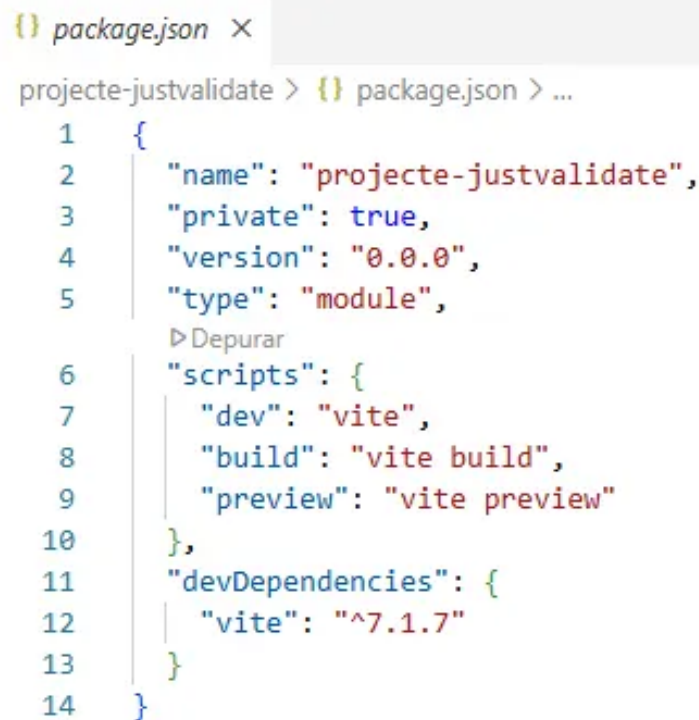
Un cop tinguem el projecte de Vite creat i les dependències instal·lades, hem d'afegir JustValidate al projecte. Dins de la ruta del projecte farem:

```
1 npm install just-validate
```

Al fer l'acció anterior, el que estem fent, a part d'instal·lar l'eina que necessitem, és escriure en el fitxer on hi ha les referències del projecte, `package.json`, que aquest projecte requereix l'eina `just-validate`.

Fixem-nos com teníem l'arxiu `package.json` abans d'instal·lar `just-validate` (figura 5.6):

FIGURA 4.6. fitxer `package.json` abans d'instal·lar "just-validate".



```
{ package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   }
14 }
```

I tot seguit el mateix arxiu just després d'instal·lar `just-validate` (figura 5.7):

FIGURA 4.7. fitxer package.json després d'instal·lar "just-validate".

```
{} package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   },
14   "dependencies": {
15     "just-validate": "^4.3.0"
16   }
17 }
18
```

L'arxiu **package.json** és un fitxer essencial en projectes de Node.js que conté informació sobre el projecte, com el nom, la versió i les dependències que necessita per funcionar. Aquestes dependències són altres paquets o llibreries que el projecte utilitza, i estan especificades en el camp `dependencies` o `devDependencies`. Quan executeu el comandament `npm install`, Node Package Manager (NPM) llegeix el `package.json`, descarrega i instal·la totes les dependències esmentades a les versions especificades, assegurant-se que el projecte tingui totes les llibreries necessàries per funcionar correctament.

Creació del formulari i les regles de validació

El formulari de validació, `index.html` per exemple, serà exactament el mateix que en l'exemple anterior. Ara, però, caldrà agafar el punt d'entrada de la lògica que serà el fitxer `main.js`.

Fixem-nos que ara el fitxer `main.js` l'afegim dins de la carpeta `src/` seguint l'estructura que Vite crea per defecte:

```
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7   </head>
8   <body>
9     <div class="container">
10      <h3>Formulari de Contacte</h3>
11      <form id="formulariContacte">
12        <div class="form-group">
13          <label for="nom">Nom complet:</label>
14          <input type="text" id="nom" name="nom" />
15        </div>
16        <div class="form-group">
17          <label for="email">Correu electrònic:</label>
18          <input type="email" id="email" name="email" />
19        </div>
20        <div class="form-group">
21          <label for="edat">Edat:</label>
22          <input type="number" id="edat" name="edat" />
23        </div>
24        <button type="submit">Enviar</button>
25      </form>
26      <p id="missatge"></p>
27    </div>
28    <script type="module" src="src/main.js"></script>
29  </body>
30 </html>
```

Ara escrivim les regles de validació a main.js:

```
1 import JustValidate from 'just-validate';
2 import "./style.css";
3
4 const validator = new JustValidate('#formulariContacte');
5
6 validator
7   .addField('#nom', [
8     { rule: 'required', errorMessage: 'El nom és obligatori' },
9     { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 carà
      cters' },
10  ])
11   .addField('#email', [
12     { rule: 'required', errorMessage: 'El correu és obligatori'
      },
13     { rule: 'email', errorMessage: 'El correu no és vàlid' },
14  ])
15   .addField('#edat', [
16     { rule: 'required', errorMessage: "L'edat és obligatòria" },
17     { rule: 'number', errorMessage: 'Només es permeten números'
      },
18     { rule: 'minNumber', value: 18, errorMessage: "Has de ser
      major d'edat" },
19  ])
20   .onSuccess(() => {
21     document.getElementById('missatge').textContent = 'Formulari
      correcte!';
22   });
```

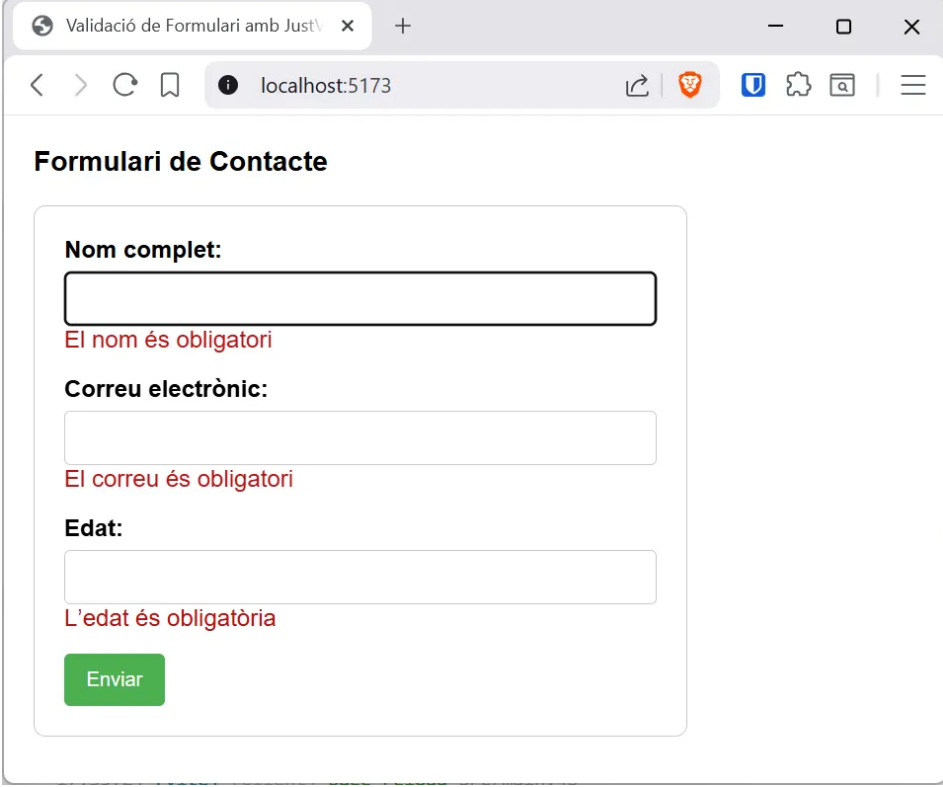
I afegirem els estils a l'arxiu `style.css`:

```
1  body {
2    font-family: Arial, sans-serif;
3    margin: 20px;
4  }
5
6  form {
7    max-width: 400px;
8    padding: 20px;
9    border: 1px solid #ccc;
10   border-radius: 8px;
11 }
12
13 .form-group {
14   margin-bottom: 15px;
15 }
16
17 label {
18   display: block;
19   margin-bottom: 5px;
20   font-weight: bold;
21 }
22
23 input[type='text'],
24 input[type='number'],
25 input[type='email'] {
26   width: 100%;
27   padding: 10px;
28   border: 1px solid #ccc;
29   border-radius: 4px;
30   box-sizing: border-box;
31 }
32
33 button {
34   background-color: #4CAF50;
35   color: white;
36   padding: 10px 15px;
37   border: none;
38   border-radius: 4px;
39   cursor: pointer;
40 }
41
42 .js-validate-error-label {
43   color: red;
44   font-size: 0.9em;
45   margin-top: 5px;
46 }
47
48 .is-invalid {
49   border-color: red;
50 }
```

Execució del projecte

Un cop tinguem tots els fitxers configurats, executarem el servidor de desenvolupament amb **Vite** (figura 4.8):

```
1  npm run dev
```

FIGURA 4.8. Execució del formulari amb validació de camps

The screenshot shows a web browser window with the title "Validació de Formulari amb Just". The address bar displays "localhost:5173". The page content is titled "Formulari de Contacte" and contains three input fields, each with a red validation message below it:

- Nom complet:** The input field is empty, and the message "El nom és obligatori" is displayed in red.
- Correu electrònic:** The input field is empty, and the message "El correu és obligatori" is displayed in red.
- Edat:** The input field is empty, and the message "L'edat és obligatòria" is displayed in red.

At the bottom of the form is a green button labeled "Enviar".

5. Framework Express. Formularis: enviament de dades

En aquesta lliçó ens endinsarem en el món dels *frameworks* per al desenvolupament de *backend* en JavaScript amb ExpressJS (o simplement, Express). Aquest framework ens permetrà crear aplicacions web de manera eficient i senzilla. A més, aprendrem a crear un servidor que sigui capaç de rebre dades des de formularis HTML, i a emmagatzemar-les en una base de dades SQLite.

5.1 Framework Express

ExpressJS és un *framework* que ens permet gestionar rutes, peticions HTTP i facilitar la creació d'aplicacions web i les API. També ens ajuda a estalviar temps a l'hora de construir un servidor, ja que ens ofereix una sintaxi neta i funcional, així com un conjunt de mòduls per a diverses operacions típiques en aplicacions web.

Per què introduïm ExpressJS en un curs de desenvolupament web del costat del client?

Tot i que aquest mòdul se centra en el desenvolupament web **del costat del client**, és essencial que els desenvolupadors entenguem com es comuniquen les aplicacions amb el **costat del servidor**. En el món real, les aplicacions web no treballen de manera aïllada: gairebé sempre necessiten **enviar i rebre dades** a través d'una API o un servidor *backend*.

Aprendre els fonaments d'ExpressJS ens permet:

- **Comprendre el cicle complet d'una aplicació web**, des de la captura de dades amb formularis fins al seu processament i emmagatzematge al servidor.
- **Treballar amb peticions HTTP (GET, POST)** i entendre com s'intercanvia informació entre el navegador i el servidor.
- **Experimentar amb un *backend* lleuger i accessible**, escrit també en JavaScript, que no requereix grans coneixements previs de programació del costat del servidor.
- **Desenvolupar autonomia i visió global** del funcionament d'un projecte web, millorant la seva capacitat per integrar el *frontend* amb serveis reals.
- **Preparar-se per futurs entorns professionals**, on la comunicació entre el client i el servidor és constant i indispensable.

A més, ExpressJS és un *framework* ideal per a introduir aquests conceptes, ja que és **simple, ràpid i compatible** amb els coneixements que els alumnes ja tenen de JavaScript. No es tracta de convertir-nos en desenvolupadors *backend*, sinó de dotar-nos **d'una base pràctica** que ens permeti entendre i provar la part del servidor amb eines properes i actuals.

5.1.1 Què és ExpressJS?

ExpressJS és un *framework* minimalista per a **Node.js** que facilita crear **servidors web i les API**. Proporciona eines senzilles per:

- Definir **rutes** (els URL) i respondre a **peticions HTTP** (GET, POST, PUT, DELETE...).
- Aplicar **middleware** (funcions que processen peticions abans d'arribar a la ruta final).
- Servir **fitxers estàtics** (HTML, CSS, JS, imatges).
- Gestionar **errors** de manera coherent.

Pensem en **ExpressJS** com una capa molt fina a sobre del servidor HTTP de Node que ens dona una sintaxi neta i unes bones pràctiques per construir aplicacions web.

5.1.2 Per a què serveix i com funciona? Alguns conceptes claus

Per a nosaltres, en aquest mòdul, ExpressJS ens servirà sobretot per **rebre dades des de formularis i respondre** al client amb confirmacions o dades.

Però també serveix per més coses, per exemple:

- **API REST** per comunicar el *frontend* amb dades (JSON).
- **Recepció de formularis** i processament de dades.
- **Renderitzat del costat del servidor** (SSR) amb motors de plantilles (EJS, Pug...), si cal.
- **Serveis lleugers**: *proxys*, *webhooks*, passarel·les, microserveis.

Procés de **funcionament** d'ExpressJS:

1. El navegador o el client fa una **petició HTTP** (ex.: GET / o POST /contacte).

2. ExpressJS passa la petició per una **cadena de middleware** (per exemple, per llegir el cos, fer logs, comprovar permisos...).
3. Una **ruta** coincideix amb el camí i el mètode (`app.get('/path', handler)`).
4. El **handler** (funció) construeix i envia una **resposta** (`res.send(...)`, `res.json(...)`, `res.status(...).send(...)`).

Conceptes clau relacionats amb ExpressJS:

- **Ruta:** associació entre un **mètode HTTP** (GET/POST/...) i un **camí** (`/`, `/api/notes`, `/users/:id`).
- **Middleware:** funció (`req`, `res`, `next`) que pot:
 - llegir/modificar `req` o `res`.
 - tallar el flux responent (`res.send(...)`).
 - o continuar amb `next()`.
- **req / res:** objectes de **sol·licitud** i **resposta** de Node/Express.
- **Fitxers estàtics:** carpeta (p. ex. `public/`) que Express serveix automàticament (`app.use(express.static('public'))`).
- **Parsers:** per llegir **JSON** i **formularis** (`express.json()`, `express.urlencoded()`).

ExpressJS ja incorpora **parsers per a JSON i formularis URL-encoded**. En materials antics es feia servir `body-parser`; actualment el més simple és `express.json()` i `express.urlencoded()`.

5.1.3 Exemple pas a pas: Mini servidor amb rutes GET/POST

En aquest exemple, crearem un mini servidor utilitzant ExpressJS que serà capaç de gestionar rutes HTTP de tipus GET i POST. Implementarem una ruta GET per obtenir dades enviades per URL, com ara un paràmetre de consulta, i una ruta POST per rebre dades des de formularis o peticions en format JSON. A més, aprendrem a servir fitxers estàtics, com pàgines HTML, que podrem provar des del navegador. Aquest servidor també inclourà el maneig bàsic d'errors per garantir que les rutes no trobades i les possibles fallades siguin gestionades correctament. A través d'aquest exercici, entendrem com funcionen les comunicacions entre el client i el servidor en una aplicació web.

Objectiu:

- Arrencar un servidor ExpressJS.

- Definir rutes **GET** i **POST**.
- Llegir dades del **query string** i del **cos** (JSON/formulari).
- Servir fitxers estàtics

Servir fitxers estàtics ens servirà per a l'apartat que trobareu més avall: "Formularis: enviament de dades al servidor".

Requisits:

- **Node.js** instal·lat (LTS recomanat).
- Un editor (VS Code, etc.).

Pas 1) Crear projecte

Per començar amb el projecte, el primer pas serà configurar l'entorn de treball. Crearem una nova carpeta per al nostre projecte i inicialitzarem un projecte Node.js utilitzant npm. A continuació, instal·larem Express, que serà el *framework* que utilitzarem per gestionar les rutes i les peticions HTTP. També tindrem l'opció d'instal·lar Nodemon, una eina útil que automàticament reinicia el servidor quan realitzem canvis al codi. Aquest pas ens permetrà tenir la infraestructura bàsica preparada per començar a desenvolupar les funcionalitats del servidor.

```
1 mkdir projecte-express
2 cd projecte-express
3 npm init -y
4 npm install express
```

Opcionalment, podem instal·lar **nodemon** que és una eina que reinicia el servidor quan detecta que s'han fet canvis al codi:

```
1 npm install -D nodemon
```

Al `package.json`, afegirem els `scripts` `dev` i `start`:

```
1 {
2   "name": "projecte-express",
3   "version": "1.0.0",
4   "description": "",
5   "main": "app.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1",
8     "dev": "nodemon app.js",
9     "start": "node app.js"
10  },
11  "keywords": [],
12  "author": "",
13  "license": "ISC",
14  "dependencies": {
15    "express": "^5.1.0"
16  },
17  "devDependencies": {
18    "nodemon": "^3.1.10"
19  }
20 }
```

Els *scripts* són comandes de **npm** (Node Package Manager) per a diferents tasques relacionades amb el projecte. En aquest cas:

- **“dev”**: **“nodemon app.js”**: Aquest script serveix per executar l'aplicació en mode de desenvolupament, utilitzant **Nodemon**. Nodemon monitora els arxius del projecte i reinicia l'aplicació automàticament cada vegada que detecta un canvi en els arxius (com per exemple un canvi en el codi).
- **“start”**: **“node app.js”**: Aquest script és per executar l'aplicació en producció o en un entorn on no es necessita reinici automàtic com en el cas de Nodemon. Utilitza el comandament **Node.js** per executar el fitxer principal (app.js).

Pas 2) Estructura bàsica de carpetes

Un cop configurat el projecte, organitzarem els fitxers en una estructura senzilla per facilitar el desenvolupament i la gestió del codi. La carpeta principal del projecte contindrà el fitxer app.js, que serà el punt d'entrada de l'aplicació, i una carpeta public/ on guardarem els fitxers estàtics com HTML, CSS i altres recursos que el servidor servirà als usuaris. Aquesta estructura ens ajudarà a mantenir el codi organitzat i permetrà una fàcil acció en el desenvolupament i les proves del projecte:

```
1 projecte-express/  
2 | app.js  
3 |-- public/  
4 |   |-- index.html  
5 |   |-- style.css
```

Crearem un public/index.html simple (ens servirà per provar-ho des del navegador):

```
1 <!DOCTYPE html>  
2 <html lang="ca">  
3   <head>  
4     <meta charset="utf-8" />  
5     <title>Projecte Express</title>  
6   </head>  
7   <body>  
8     <h1>Hola des d'Express!</h1>  
9     <p>  
10       Prova <em>/api/hola?nom=Maria</em> o envia un POST a  
11       <em>/api/eco</em>.  
12     </p>  
13   </body>  
14 </html>
```

Pas 3) Codi del servidor: app.js

Ara que tenim la infraestructura bàsica preparada, crearem el fitxer app.js, que serà el cor del nostre servidor Express. En aquest fitxer, definirem les rutes i

configurarem els diferents elements necessaris per al funcionament del servidor, com els *middleware* per processar les peticions, la gestió de fitxers estàtics i les respostes de les rutes. També inclourem les rutes de demostració, com la ruta GET per obtenir un missatge personalitzat i la ruta POST per retornar les dades rebudes. Aquest codi serà la base sobre la qual construïrem la funcionalitat del servidor:

```
1  const express = require("express");
2
3  const app = express();
4  const PORT = 3000;
5
6  // 1) Middleware de parsers
7  app.use(express.json()); // per cosos JSON (
   application/json)
8  app.use(express.urlencoded({ extended: true })); // per
   formularis HTML (application/x-www-form-urlencoded)
9
10 // 2) Servir fitxers estàtics (public/)
11 app.use(express.static('public'));
12
13 // 3) Rutes de demostració
14
15 // GET /api/hola?nom=Maria -> { missatge: "Hola, Maria!" }
16 app.get('/api/hola', (req, res) => {
17   const nom = req.query.nom || 'món';
18   res.json({ missatge: 'Hola, ${nom}!' });
19 });
20
21 // POST /api/eco (rep JSON o formulari, i el retorna)
22 app.post('/api/eco', (req, res) => {
23   // req.body conté les dades del client (JSON o formulari)
24   res.status(201).json({
25     rebut: true,
26     dades: req.body
27   });
28 });
29
30 // 4) Ruta 404 per a camins no existents d'API
31 app.use('/api', (req, res) => {
32   res.status(404).json({ error: 'Ruta d'API no trobada' });
33 });
34
35 // 5) Middleware de gestió d'errors (exemple bàsic)
36 app.use((err, req, res, next) => {
37   console.error('Error:', err);
38   res.status(500).json({ error: 'Error intern del servidor' });
39 });
40
41 // 6) Posar el servidor en marxa
42 app.listen(PORT, () => {
43   console.log('Servidor escoltant a http://localhost:${PORT}');
44 });
```

Pas 4) Executar el projecte

Un cop tinguem el codi del servidor creat, el següent pas serà executar-lo i posar-lo en marxa. Per fer-ho, utilitzarem l'script de desenvolupament amb Nodemon, que reiniciarà automàticament el servidor cada cop que fem canvis en el codi. Un

cop el servidor estigui en marxa, podrem obrir el navegador i provar les rutes definides, com la pàgina estàtica inicial i les API que hem creat, comprovant així que tot funcioni correctament. Aquest procés ens permetrà veure els resultats en temps real i verificar que el servidor estigui responnent a les peticions com esperem:

```
1 npm run dev
```

Obrirem el navegador a: <http://localhost:3000>

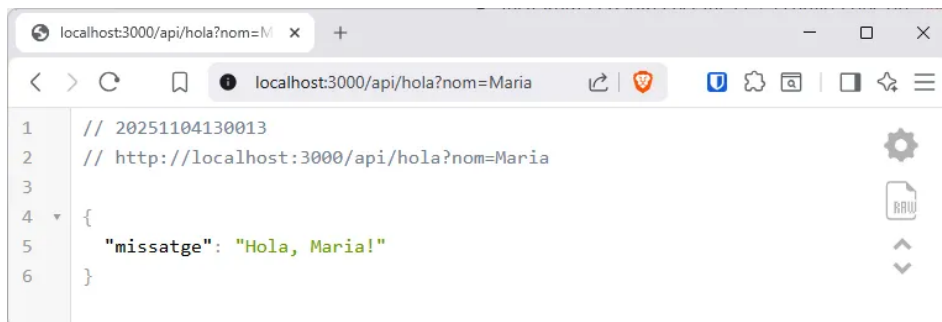
- Visitarem la pàgina estàtica (/) servida des de public/ (figura 5.1).

FIGURA 5.1. Execució del projecte



- Provarem l'API:
 - **GET** (en la barra del navegador): <http://localhost:3000/api/hola?nom=Maria> (figura 5.2)

FIGURA 5.2. Petició API amb Express (mètode GET)



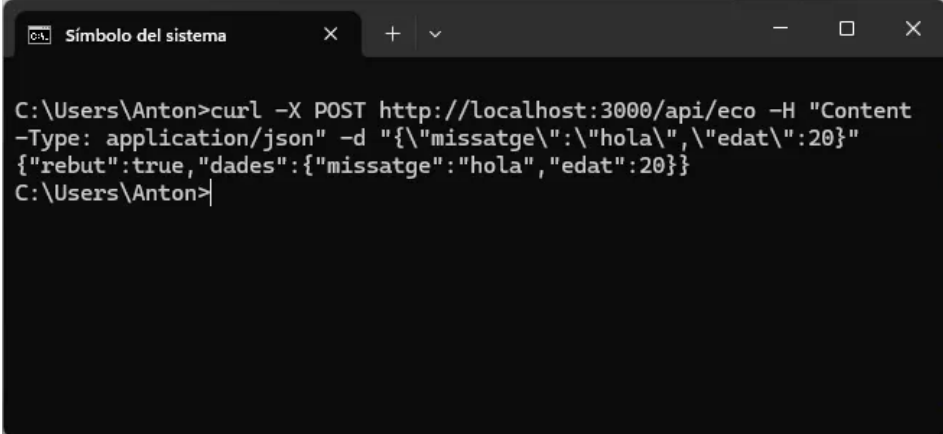
- **POST** (des de la consola amb cURL, o des de l'aplicació Postman que s'ha d'instal·lar) (figura 5.3):

```
1 curl -X POST http://localhost:3000/api/eco -H "Content-Type: application/json" -d '{"missatge":"hola","edat":20}'
```

Resposta esperada (figura 5.3):

```
1 {  
2     "rebut": true,  
3     "dades": { "missatge": "hola", "edat": 20 }  
4 }
```

FIGURA 5.3. Resposta rebuda de l'API



```
C:\Users\Anton>curl -X POST http://localhost:3000/api/eco -H "Content  
-Type: application/json" -d "{\"missatge\":\"hola\",\"edat\":20}"  
{"rebut":true,"dades":{"missatge":"hola","edat":20}}  
C:\Users\Anton>
```

cURL és una eina de línia d'ordres que permet realitzar peticions HTTP a un servidor, ideal per provar les API i serveis web. Es pot utilitzar per enviar peticions GET, POST, PUT, DELETE, entre altres, i permet especificar capçaleres, cos de la petició i altres paràmetres.

Postman és una aplicació gràfica que facilita el testatge de les API. Ofereix una interfície d'usuari més amigable, on pots crear peticions HTTP de manera visual, gestionar entorns, veure les respostes en diferents formats i automatitzar tests per validar el comportament de l'API. Ambdues eines són molt útils per desenvolupar i depurar serveis web.

5.2 Formularis: enviament de dades al servidor

Els **formularis HTML** són una eina fonamental per a capturar dades dels usuaris. En aquesta secció, veurem com enviar aquestes dades al servidor utilitzant el mètode POST i com ExpressJS pot gestionar aquesta informació. Per facilitar el procés, utilitzarem la llibreria **body-parser** per analitzar les dades de les peticions POST.

Quan l'usuari envia un formulari, el navegador crea una **petició HTTP** (normalment de tipus POST) i envia les dades al servidor.

El **servidor Express** rep aquestes dades, les processa i pot:

- Validar-les
- Desar-les a una base de dades (o a un fitxer, o qualsevol altra cosa)

- Retornar una resposta al client

Aquesta secció ens permet **tancar el cicle complet**: *Frontend* (formulari) → Servidor (Express) → Emmagatzematge (SQLite) → Resposta al client.

5.2.1 Teoria bàsica: com funciona l'enviament de dades

Quan enviem un formulari com aquest...

```
1 <form action="/formulari" method="POST">
2   <label>Nom: <input type="text" name="nom" /></label>
3   <label>Correu: <input type="email" name="correu" /></label>
4   <button type="submit">Enviar</button>
5 </form>
```

..., el navegador fa una petició:

```
1 POST /formulari HTTP/1.1
2 Content-Type: application/x-www-form-urlencoded
3 Body: nom=Anna&correu=anna%40mail.com
```

A Express podem llegir aquest contingut amb:

```
1 app.use(express.urlencoded({ extended: true }));
```

Si l'enviament fos en **JSON**, faríem servir:

```
1 app.use(express.json());
```

5.2.2 Exemple pràctic: Formulari que envia dades al servidor

En aquest exemple pràctic, crearem un formulari HTML simple que permetrà als usuaris enviar dades al servidor mitjançant el mètode POST. Aquest formulari recollirà informació com el nom i el correu electrònic, i un cop enviades, les dades seran processades pel servidor Express. El servidor les emmagatzemarà a una base de dades SQLite i retornarà una resposta al client per confirmar que les dades s'han rebut correctament. A través d'aquest exercici, veurem com gestionar dades de formularis en una aplicació web i com interactuar amb una base de dades des del servidor.

Estructura del projecte

Crearem la següent estructura per desenvolupar aquest exemple:

```
1 formulari-express/  
2 | app.js  
3 | dades.db ← base de dades SQLite  
4 |-- public/  
5   |-- index.html  
6   |-- style.css
```

Instal·lacions necessàries:

```
1 mkdir formulari-express  
2 cd formulari-express  
3 npm init -y  
4 npm install express sqlite3
```

(Opcional: `npm install -D nodemon` i afegir l'script "dev": "nodemon app.js")

Codi del servidor (app.js)

En aquest apartat, escriurem el codi del servidor en el fitxer `app.js`, on configurarem Express per gestionar les peticions HTTP.

Primer, definirem els *middleware* necessaris per processar les dades del formulari i del JSON, així com per servir els fitxers estàtics. Després, establim la connexió a la base de dades SQLite i crearem una taula per emmagatzemar les dades del formulari. Finalment, implementarem una ruta POST per rebre les dades del formulari, validar-les i desar-les a la base de dades, així com una ruta GET per obtenir els contactes desats.

Aquest codi serà el nucli del nostre servidor i gestionarà la comunicació entre el client i la base de dades:

```
1 const express = require("express");  
2 const sqlite3 = require("sqlite3");  
3  
4 const app = express();  
5 const PORT = 3000;  
6  
7 // Middleware per llegir formularis i JSON  
8 app.use(express.urlencoded({ extended: true }));  
9 app.use(express.json());  
10  
11 // Servir fitxers estàtics  
12 app.use(express.static('public'));  
13  
14 // Connexió a la base de dades SQLite  
15 const db = new sqlite3.Database("./dades.db", (err) => {  
16   if (err) console.error(err.message);  
17   else console.log("Connexió a SQLite establerta.");  
18 });  
19  
20 // Crear la taula de contactes si no existeix  
21 db.run(  
22   'CREATE TABLE IF NOT EXISTS contactes (  
23     id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
24     nom TEXT NOT NULL,  
25     correu TEXT NOT NULL  
26   )'  
27 );  
28  
29 // Ruta POST per rebre les dades del formulari  
30 app.post('/formulari', async (req, res) => {  
31   const { nom, correu } = req.body;  
32  
33   if (!nom || !correu) {  
34     return res.status(400).send('Falten camps obligatoris');  
35   }  
36  
37   db.run('INSERT INTO contactes (nom, correu) VALUES (?, ?)', [  
38     nom, correu]);  
39  
40   res.send('<h2>Dades rebudes correctament</h2>  
41     <p>Nom: ${nom}</p>  
42     <p>Correu: ${correu}</p>  
43     <a href="/">Tornar</a>');  
44  
45   });  
46  
47 // Obtener tots els contactes (GET /contactes)  
48 app.get("/contactes", (req, res) => {  
49   db.all("SELECT * FROM contactes", [], (err, rows) => {  
50     if (err) return res.status(500).json({ error: err.  
51       message });  
52     res.json(rows);  
53   });  
54  
55   });  
56  
57 // Posar en marxa el servidor  
58 app.listen(PORT, () => {  
59   console.log('Servidor actiu a http://localhost:${PORT}');  
60 });
```

Formulari HTML (public/index.html)

Ara crearem el formulari HTML que els usuaris utilitzaran per enviar les seves dades al servidor. El formulari permetrà introduir el nom i el correu electrònic, que seran enviats mitjançant una petició POST al servidor Express. Inclourem les etiquetes bàsiques de formulari, com input per als camps de text i button per enviar la informació.

Aquest formulari es presentarà en una pàgina web senzilla i utilitzarà l'estil bàsic definit en un fitxer CSS per millorar la seva visualització.

Quan els usuaris enviïn el formulari, les dades seran enviades al servidor per ser processades i emmagatzemades:

```
1 <!DOCTYPE html>  
2 <html lang="ca">  
3 <head>  
4   <meta charset="utf-8" />  
5   <title>Formulari Express</title>  
6   <link rel="stylesheet" href="style.css">  
7 </head>  
8 <body>
```

```
9    <h1>Enviament de dades al servidor</h1>
10
11    <form action="/formulari" method="POST">
12        <label>Nom:<br>
13            <input type="text" name="nom" required>
14        </label><br><br>
15
16        <label>Correu:<br>
17            <input type="email" name="correu" required>
18        </label><br><br>
19
20        <button type="submit">Enviar</button>
21    </form>
22
23    <hr>
24
25    <p><a href="/contactes">Veure tots els contactes (JSON)</a></p>
26</body>
27</html>
```

CSS bàsic (public/style.css)

Afegirem uns estils bàsics per millorar l'experiència de l'usuari:

```
1  body {
2      font-family: sans-serif;
3      max-width: 600px;
4      margin: 40px auto;
5      line-height: 1.6;
6  }
7
8  label {
9      display: block;
10 }
11
12 input, button {
13     padding: 6px;
14     font-size: 1rem;
15 }
16
17 button {
18     background: #007bff;
19     color: white;
20     border: none;
21     cursor: pointer;
22     border-radius: 4px;
23 }
24
25 button:hover {
26     background: #0056b3;
27 }
```

Proves

Tot seguit realitzarem les proves per verificar que el servidor funciona correctament. Un cop tinguem el codi del servidor i el formulari configurats, provarem la

funcionalitat enviant dades des del formulari HTML al servidor i comprovarem que aquestes dades es desin correctament a la base de dades. Utilitzarem el navegador per omplir el formulari i enviar les dades, i després accedirem a les rutes definides per veure les dades desades. També comprovarem les respostes de les peticions GET i POST per assegurar-nos que el sistema estigui processant la informació com es preveu.

Pas 1) Executarem:

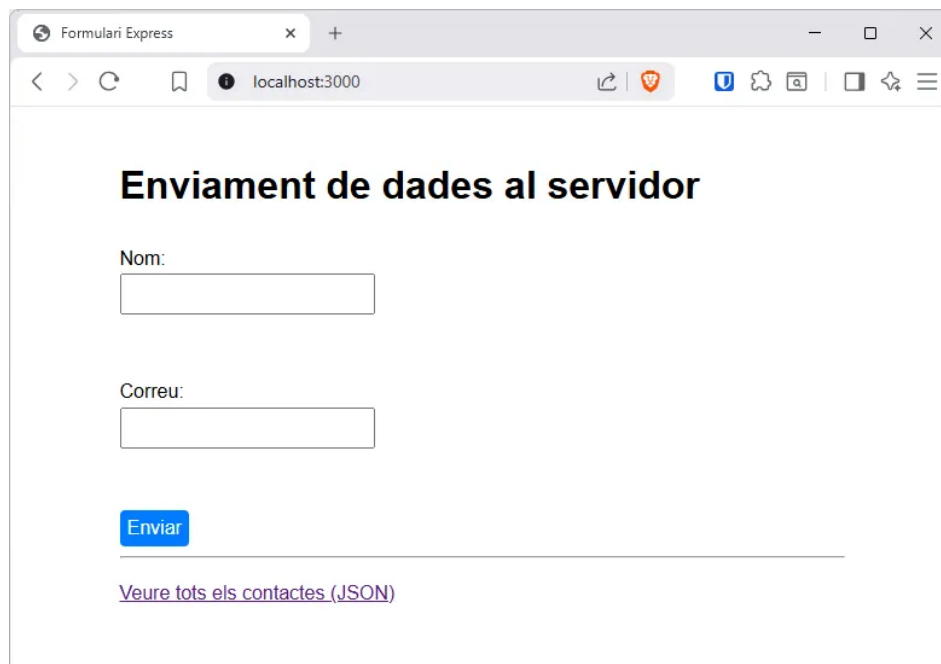
```
1 node app.js
```

O, si hem creat l'script dev, farem:

```
1 npm run dev
```

Pas 2) Obrim <http://localhost:3000> (figura 5.4).

FIGURA 5.4. Formulari



The screenshot shows a web browser window with the title 'Formulari Express'. The address bar shows 'localhost:3000'. The main content area has the heading 'Enviament de dades al servidor'. Below the heading are two input fields: 'Nom:' and 'Correu:'. Below the 'Correu:' field is a blue button labeled 'Enviar'. At the bottom of the form is a link that says 'Veure tots els contactes (JSON)'.

Pas 3) Omplim dades i enviem el formulari (figura 5.5).

FIGURA 5.5. Omplim les dades del formulari

Formulari Express

localhost:3000

Enviament de dades al servidor

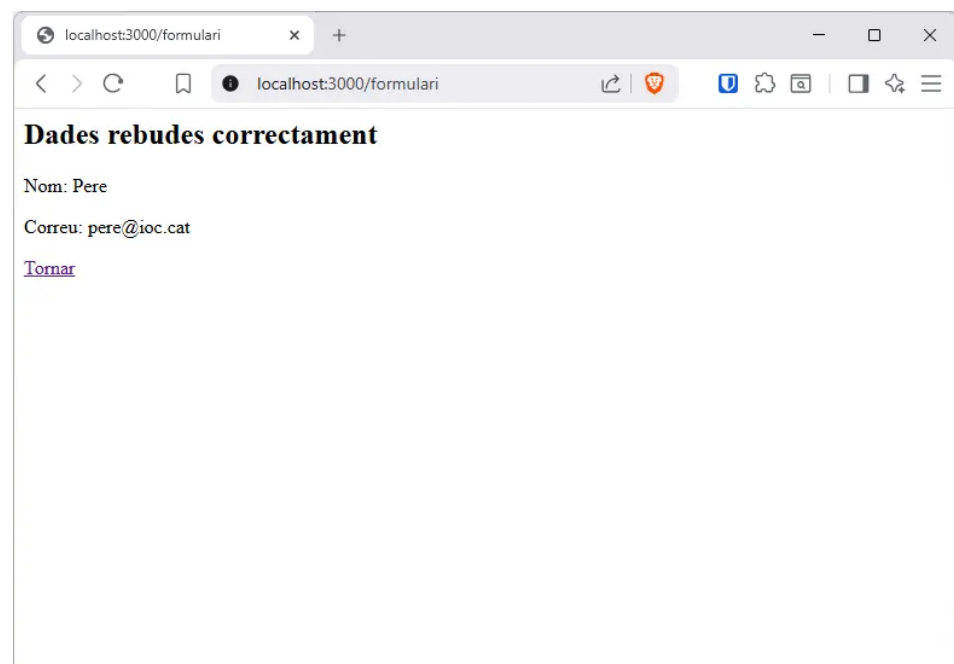
Nom:

Correu:

[Enviar](#)

[Veure tots els contactes \(JSON\)](#)

Pas 4) Express mostrarà una pàgina amb el missatge de confirmació (figura 5.6).

FIGURA 5.6. Resposta del servidor

localhost:3000/formulari

localhost:3000/formulari

Dades rebudes correctament

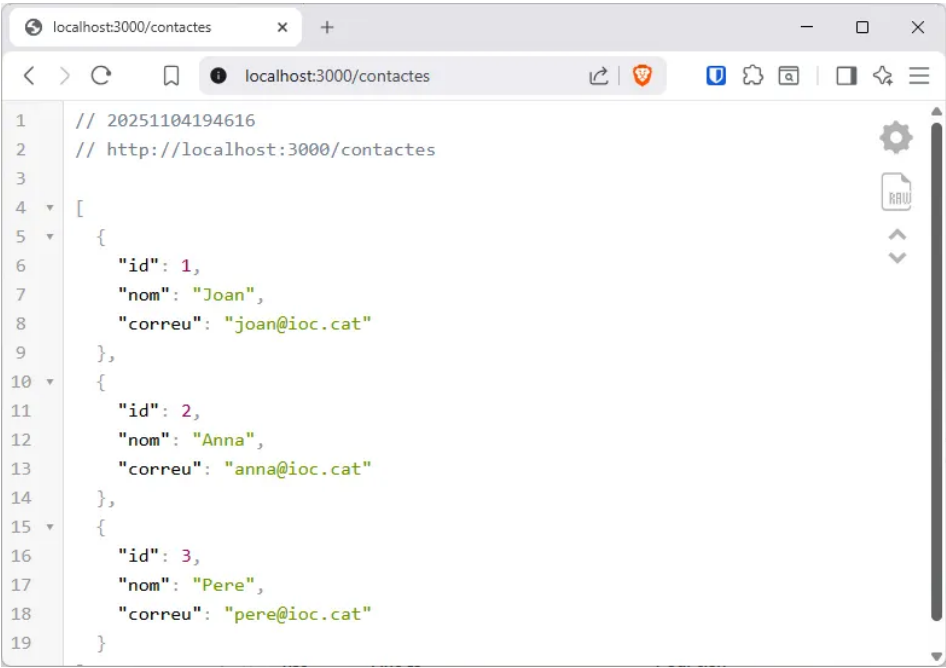
Nom: Pere

Correu: pere@ioc.cat

[Tornar](#)

Pas 5) Provarem <http://localhost:3000/contactes> per veure les dades desades a la base SQLite (figura 5.7).

FIGURA 5.7. Informació provinent de la base de dades



Explicació pas a pas

Si hem provat la nostra aplicació i hem enviat les dades a través del formulari, la taula 5.1 mostra el que internament ocorre:

TAULA 5.1. Passos comunicació formulari amb servidor

Pas	Què fa?	Codi clau
1	El navegador envia una petició POST amb les dades del formulari.	<form action="/formulari" method="POST">
2	Express intercepta la petició i llegeix el cos.	app.use(express.urlencoded(...))
3	El servidor processa les dades i les desa.	db.run('INSERT INTO contactes...')
4	Express retorna una resposta HTML al client.	res.send(...)
5	L'usuari veu un missatge de confirmació i pot consultar les dades desades.	/contactes